

claude-windows / 662b86ab-d2e3-4c02-8a0d-ec7a9071aba8

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\662b86ab-d2e3-4c02-8a0d-ec7a9071aba8\subagents\workflows\wf_34671ac1-e8e\agent-a48c157ab9e054fa6.jsonl`  
- SHA-256: `57e67c27390cca5b0194421fb90b8b14fcc87fec3d6e367844d6ff8e4eea9088`  
- Source modified: `2026-06-21T10:22:06+00:00`  
- Imported at: `2026-07-05T16:48:27+00:00`  
- project: `wf_34671ac1-e8e`  
- session_id: `662b86ab-d2e3-4c02-8a0d-ec7a9071aba8`
```

Transcript

1. user (2026-06-21T10:18:25.335Z)

Review the COMPUTE_DECOUPLED_SERVING **M5** change just committed on branch feat/serving-m5-truly-local-checks (HEAD). Inspect it with: `git diff origin/master...HEAD`, and read the full files:

- backend/config/startup.py (NEW ? the per-profile fail/warn-fast checks)
- backend/config/presets.py (added probe_cuda_available)
- backend/config/__init__.py (exports)
- backend/main.py (server main(): enforce after global_config load; sys.exit(1) on fatal)
- backend/worker/__main__.py (_run_startup_checks helper; called in cmd_infer + cmd_train after _apply_overrides; returns rc 2 on fatal)

- tests/test_startup_checks.py (NEW, L1) + tests/test_profile_parity.py (NEW, L4)

The contract (must hold): DEFAULT-OFF ? when deployment.profile=="" NOTHING changes vs pre-M5 (byte-identical). Only an explicitly-selected profile triggers checks. cloud-serving + non-cloud storage = the ONLY fatal error (everything else is a warning) deliberately, to avoid false-positives (the CUDA probe is best-effort; ADC creds can't be confirmed offline). One isolated PR, default-OFF discipline. Report ONLY real, grounded issues (file:line). If a lens finds nothing real, return an empty findings array ? do not invent nits.

LENS = TEST RIGOR / FALSE-CONFIDENCE. The L4 parity test (tests/test_profile_parity.py) is the load-bearing decoupling proof ? scrutinise it hard. (1) Does test_inprocess_segment_ranges_identical_across_profiles actually exercise REAL windowing (trajectory_to_action_windows), or could it pass vacuously (e.g. all-empty results compared equal)? It asserts len>=1 ? is that enough? (2) Is the stub trajectory genuinely IDENTICAL across the three profile arms (same synth params / fps), so equality is meaningful and not coincidental? (3) test_worker_intervals_match_inprocess: is the worker's trajectory truly the same as the in-process run (different video_id ? does that affect windows)? Could the comparison be comparing two empty lists? (4) The signature-guard test ? is it meaningful? (5) test_startup_checks.py ? any assertion too weak, or a case that can't actually fail? Find vacuous/weak/misleading assertions.

2. assistant (2026-06-21T10:18:28.714Z)

I'll review the M5 change with the test-rigor / false-confidence lens. Let me start by pulling the latest state and inspecting the diff.

3. user (2026-06-21T10:18:33.184Z)

```
---BRANCH---
```

```
feat/serving-m5-truly-local-checks
```

```
---HEAD---
```

```
6f78acd feat(serving): M5 ? per-profile startup checks + L4 profile-parity test (default-OFF)
```

```
---vs master---
```

```
6f78acd feat(serving): M5 ? per-profile startup checks + L4 profile-parity test (default-OFF)
```

4. user (2026-06-21T10:18:33.396Z)

backend/config/__init__.py	13 +-
backend/config/presets.py	12 ++
backend/config/startup.py	179 ++++++
backend/main.py	18 +-
backend/worker/__main__.py	23 ++-
.../runlogs/RUNLOG_truly-local_TEMPLATE.md	49 ++++++
tests/test_profile_parity.py	139 ++++++
tests/test_startup_checks.py	181 ++++++

8 files changed, 611 insertions(+), 3 deletions(-)

5. assistant (2026-06-21T10:18:35.913Z)

Let me read all the changed files in full.

6. user (2026-06-21T10:18:37.132Z)

```

1      """Per-profile startup checks (COMPUTE_DECOUPLED_SERVING M5).
2
3      A deployment profile (`truly-local` / `cloud-serving` / `cpu-mock`) implies a
coherent
4      runtime. These checks fail/warn fast at process startup ? a loud signal before any
5      (expensive) job runs ? when the active profile is inconsistent with the environment:
6
7      * cloud-serving with a non-cloud storage backend ? a hard error (the one
config bug we
8      can detect with certainty + zero false-positive risk: a cloud profile that can't
reach a
9      bucket can never serve). This is the README §4.3 "loud error before any job".
10     * a GPU mismatch (e.g. `device='cuda'` on a box where CUDA isn't visible, or
`device='auto'`
11     falling back to CPU) ? a warning, never an error: the probe is best-effort
(torch may live
12     only in the detector subprocess env ? see `presets.probe_cuda_available`), and the
detector's
13     own healthcheck is the real authority. A warning surfaces it early without false-
failing.
14     * creds that can't be confirmed (no `GOOGLE_APPLICATION_CREDENTIALS` / not on
Cloud Run) ?
15     a warning: Application Default Credentials can come from the metadata server /
gcloud, which
16     we can't probe offline, so the storage backend's own init is the authority; we just
flag it.
17
18     DEFAULT-OFF: with no profile selected (`profile == ""`) this is a strict no-op
? exactly
19     the pre-M5 behaviour. Only an explicitly-selected profile triggers any check (mirrors
the loader,
20     which applies a preset only for an explicit profile). So nothing changes for today's
manual-knob
21     deployments.
22     """
23
24     from __future__ import annotations
25
26     import logging
27     import os
28     from dataclasses import dataclass
29     from typing import Any, Mapping, Optional
30
31     LEVEL_ERROR = "error"
32     LEVEL_WARN = "warn"
33
34     # Storage backends that count as "a cloud bucket" for cloud-serving coherence.
35     _CLOUD_STORAGE = ("gcs", "s3")
36     # Env signals that imply Google ADC might resolve without an explicit creds file.

```

```

37     _GCP_ENV_HINTS = ("GOOGLE_APPLICATION_CREDENTIALS", "K_SERVICE", "GCE_METADATA_HOST",
38                        "CLOUDSDK_CONFIG", "GOOGLE_CLOUD_PROJECT")
39
40
41     @dataclass(frozen=True)
42     class StartupCheck:
43         """One per-profile finding. ``level`` is ``error`` (fatal) or ``warn`` (heads-
44 up)."""
45         level: str
46         code: str          # stable, greppable code (e.g. "STORAGE_INCOHERENT")
47         message: str
48
49
50     class StartupCheckError(RuntimeError):
51         """Raised by ``enforce_startup_checks`` when an active profile fails a FATAL
52 check."""
53         def __init__(self, failures: list[StartupCheck]):
54             self.failures = failures
55             super().__init__(
56                 "deployment profile startup check failed: "
57                 + "; ".join(f"[{c.code}] {c.message}" for c in failures)
58             )
59
60
61     def _creds_discoverable(env: Mapping[str, str]) -> bool:
62         """True if *some* Google ADC source is hinted by the environment. Best-effort ? a
63 False here
64         only downgrades to a WARNING, never a hard failure (the storage init is the
65 authority)."""
66         return any(env.get(k) for k in _GCP_ENV_HINTS)
67
68     def run_startup_checks(
69         config: Any,
70         *,
71         cuda_available: Optional[bool] = None,
72         env: Optional[Mapping[str, str]] = None,
73     ) -> list[StartupCheck]:
74         """Return the per-profile findings for ``config`` (empty when no profile is
75 selected).
76
77         ``cuda_available`` is the caller's best-effort GPU probe (``None`` = not probed ?
78 GPU checks
79 are skipped; the API server passes ``None``, the worker passes
80 ``presets.probe_cuda_available()``).
81 Pure: no IO, no torch import ? the caller owns the probe so this stays importable
82 everywhere.
83
84         """
85         profile = getattr(getattr(config, "deployment", None), "profile", "") or ""
86         if not profile:
87             return [] # DEFAULT-OFF: no profile ? no checks (identical to pre-M5).
88
89         e = os.environ if env is None else env
90         storage_backend = getattr(getattr(config, "storage", None), "backend", "local")
91         indexing = getattr(config, "indexing", None)
92         detector_impl = getattr(indexing, "detector_impl", "auto")
93         device = getattr(getattr(indexing, "wasb", None), "device", "cuda")
94
95         checks: list[StartupCheck] = []
96
97         if profile == "truly-local":
98             # truly-local reads bytes from the local FS or a *mounted* Drive ? a cloud
99 backend here is

```

```

93         # an inconsistent (but not fatal) choice worth surfacing.
94         if storage_backend not in ("local", "gdrive"):
95             checks.append(StartupCheck(
96                 LEVEL_WARN, "STORAGE_UNUSUAL",
97                 f"profile 'truly-local' usually uses local/gdrive storage, but
storage_backend="
98                 f"'{storage_backend}'. Reads will go to a remote backend.",
99             ))
100         # GPU heads-up (warning only ? the detector healthcheck is the authority; the
probe is
101         # best-effort and may be False even when a subprocess detector env has CUDA).
102         if cuda_available is False:
103             if device == "cuda":
104                 checks.append(StartupCheck(
105                     LEVEL_WARN, "CUDA_NOT_VISIBLE",
106                     "device='cuda' but no CUDA GPU is visible to this process. If the
box truly "
107                     "has no GPU the detector healthcheck will hard-fail ? set
indexing.wasb.device="
108                     "'auto' for the CPU fallback (M4).",
109                 ))
110             elif device == "auto":
111                 checks.append(StartupCheck(
112                     LEVEL_WARN, "CPU_FALLBACK",
113                     "device='auto' resolved to CPU (no CUDA GPU visible). Inference will
run on "
114                     "CPU ? correct, but much slower than a GPU.",
115                 ))
116
117         elif profile == "cloud-serving":
118             # The ONE fatal coherence check: a cloud profile that can't reach a bucket can't
serve.
119             if storage_backend not in _CLOUD_STORAGE:
120                 checks.append(StartupCheck(
121                     LEVEL_ERROR, "STORAGE_INCOHERENT",
122                     f"profile 'cloud-serving' requires a cloud storage backend "
123                     f"('{','.join(_CLOUD_STORAGE)}'), but storage_backend='{storage_backend}'.
"
124                     "A cloud job cannot read/write a local-only backend.",
125                 ))
126             elif storage_backend == "gcs" and not _creds_discoverable(e):
127                 checks.append(StartupCheck(
128                     LEVEL_WARN, "CREDS_UNCONFIRMED",
129                     "no GOOGLE_APPLICATION_CREDENTIALS / Cloud Run env detected; relying on
Application "
130                     "Default Credentials. The GCS backend will fail loudly at init if ADC
can't resolve.",
131                 ))
132             if device == "cpu":
133                 checks.append(StartupCheck(
134                     LEVEL_WARN, "CLOUD_CPU_DEVICE",
135                     "profile 'cloud-serving' runs on a GPU (L4) but
indexing.wasb.device='cpu' ? "
136                     "detection will not use the GPU.",
137                 ))
138             elif cuda_available is False and device in ("cuda", "auto"):
139                 checks.append(StartupCheck(
140                     LEVEL_WARN, "CLOUD_CUDA_NOT_VISIBLE",
141                     "profile 'cloud-serving' expects a GPU but no CUDA GPU is visible to
this process.",
142                 ))
143
144         elif profile == "cpu-mock":
145             # The mock profile is for CI/regression ? a real detector here defeats the
purpose.

```

```

146         if detector_impl != "stub":
147             checks.append(StartupCheck(
148                 LEVEL_WARN, "MOCK_NONSTUB_DETECTOR",
149                 f"profile 'cpu-mock' normally uses detector_impl='stub', but
detector_impl="
150                 f"'{detector_impl}'. A real detector defeats the GPU-free mock.",
151                 ))
152
153     return checks
154
155
156 def enforce_startup_checks(
157     config: Any,
158     *,
159     cuda_available: Optional[bool] = None,
160     env: Optional[Mapping[str, str]] = None,
161     raise_on_error: bool = True,
162     logger: Optional[logging.Logger] = None,
163 ) -> list[StartupCheck]:
164     """Run the checks, LOG each (errors at ERROR, warnings at WARNING), and ? unless
165     ``raise_on_error=False`` ? raise ``StartupCheckError`` if any FATAL check tripped.
166
167     Returns the full findings list (so a caller can inspect/surface them). A no-op with
no logging
168     and no raise when no profile is selected (``run_startup_checks`` returns ``[]``)."""
169     log = logger or logging.getLogger(__name__)
170     checks = run_startup_checks(config, cuda_available=cuda_available, env=env)
171     for c in checks:
172         if c.level == LEVEL_ERROR:
173             log.error("[STARTUP] %s: %s", c.code, c.message)
174         else:
175             log.warning("[STARTUP] %s: %s", c.code, c.message)
176     errors = [c for c in checks if c.level == LEVEL_ERROR]
177     if errors and raise_on_error:
178         raise StartupCheckError(errors)
179     return checks
180

```

7. user (2026-06-21T10:18:37.417Z)

```

1     """Profile-parity ? the load-bearing decoupling proof (COMPUTE_DECOUPLED_SERVING M5,
L4).
2
3     The decoupling guarantee: the same `(trajectory + config)` produces the **same candidate
4     windows + the same segment ranges** regardless of the active deployment profile /
orchestration.
5     Detection is replaced by the deterministic CPU **stub** (no GPU/WSL/network), so ANY
divergence
6     is necessarily a profile/seam leak into the pure core ? exactly what D1 forbids.
7
8     Three arms:
9     * **L4a** ? the WINDOW stage (`trajectory_to_action_windows`) is a pure, profile-
agnostic
10     staticmethod: deterministic across calls + its signature carries no
profile/compute/storage
11     parameter (so it *structurally* cannot branch on a profile).
12     * **L4b** ? the real stub-backed segmenter produces byte-identical segment ranges
under
13     `profile=truly-local` vs `profile=cpu-mock` vs no profile (the only varied knob is
the
14     compute target, which the core never reads).
15     * **L4c** ? the worker (`cmd_infer`, the cloud-serving orchestration) writes an
`intervals.csv`
16     whose ranges match the in-process truly-local run for the same stub trajectory.
17     """

```

```

18     import inspect
19     import json
20
21     from backend.config import load_config
22     from backend.infrastructure.database import Database
23     from backend.pipeline.detectors.stub_runner import StubDetectorRunner
24     from backend.pipeline.segmenters.wasb_hybrid import WasbHybridSegmenter
25
26
27     # A fixed duration so process_video's early ffprobe check passes for the fake
(undecodable) video;
28     # applied identically to every arm so it can never be the source of a divergence.
29     _DURATION = 30.0
30
31
32     def _config(tmp_path, profile):
33         """A config.json applying `profile` (or none), forced to the deterministic CPU stub
+
34         fully-local (no AI handoff) + a tmp output dir. (The stub uses its default synth
params ?
35         `indexing.stub` is not a typed knob, so it deterministically self-configures.)"""
36         data = {
37             "indexing": {"detector_impl": "stub", "skip_ai_handoff": True},
38             "output": {"output_dir": str(tmp_path / "out")},
39         }
40         if profile:
41             data["deployment"] = {"profile": profile}
42         cfg_file = tmp_path / f"config_{profile or 'none'}.json"
43         cfg_file.write_text(json.dumps(data))
44         return load_config(cfg_file)
45
46
47     def _ranges(segments):
48         """Normalise a segment list to comparable (start, end) tuples (rounded to drop float
noise)."""
49         out = []
50         for s in segments:
51             start = s.get("start_time", s.get("start"))
52             end = s.get("end_time", s.get("end"))
53             out.append((round(float(start), 3), round(float(end), 3)))
54         return out
55
56
57     def _inprocess_ranges(tmp_path, profile, monkeypatch):
58         tmp_path.mkdir(parents=True, exist_ok=True)
59         # The fake video is undecodable; pin the duration probe so the early validity check
passes.
60         # Applied to the shared module fn ? identical for every profile arm (never a
divergence source).
61
62         monkeypatch.setattr("backend.pipeline.segmenters.trajectory_hybrid.get_video_duration",
63                             lambda p: _DURATION)
64         db = Database(str(tmp_path / f"db_{profile or 'none'}.sqlite"))
65         cfg = _config(tmp_path, profile)
66         video = tmp_path / "in.mp4"
67         video.write_bytes(b"FAKEVIDEO-PARITY")
68         db.add_video("vidParity", str(video), "processed", []) # FK target for
candidate/play rows
69         seg = WasbHybridSegmenter(db, cfg)
70         result = seg.process_video("vidParity", str(video))
71         return _ranges(result)
72
73     # ----- L4a (pure WINDOW
stage)

```

```

74
75     def test_window_stage_has_no_profile_parameter():
76         """Structural guard: the WINDOW stage (`trajectory_to_action_windows`) carries no
77         profile/compute/storage parameter, so it *cannot* branch on a deployment profile ?
the
78         precise leak D1 forbids. (Determinism across profiles is exercised end-to-end in
L4b.)"""
79         sig = inspect.signature(StubDetectorRunner.trajectory_to_action_windows)
80         banned = {"profile", "compute_target", "deployment", "storage", "backend"}
81         assert not (banned & set(sig.parameters)), \
82             f"WINDOW stage must not take a profile/compute/storage param; got
{list(sig.parameters)}"
83
84
85     # ----- L4b (in-process
parity)
86
87     def test_inprocess_segment_ranges_identical_across_profiles(tmp_path, monkeypatch):
88         """Same stub trajectory + config under different profiles ? byte-identical segment
ranges.
89         The profiles differ only in compute_target/label, which the pure core never
reads."""
90         local = _inprocess_ranges(tmp_path / "a", "truly-local", monkeypatch)
91         mock = _inprocess_ranges(tmp_path / "b", "cpu-mock", monkeypatch)
92         none = _inprocess_ranges(tmp_path / "c", "", monkeypatch)
93         assert local == mock == none
94         assert len(local) >= 1 # the stub actually produced rallies (the test isn't
vacuous)
95
96
97     # ----- L4c (worker
orchestration)
98
99     def test_worker_intervals_match_inprocess(tmp_path, monkeypatch):
100         """The worker (cmd_infer ? the cloud-serving orchestration) serialises an
intervals.csv whose
101         ranges equal the in-process truly-local run for the same stub trajectory. Proves the
102         fetch/store/serialise orchestration doesn't perturb the core."""
103         from backend.worker import __main__ as wmain
104
105         # In-process reference ranges (this also pins get_video_duration for the worker run
below).
106         ref = _inprocess_ranges(tmp_path / "ref", "truly-local", monkeypatch)
107
108         # Worker path: stub + skip-AI, validators mocked (fake bytes aren't a real video).
109         class _OK:
110             passed = True
111             validator_name = "fake"
112             message = "ok"
113
114             def model_dump(self):
115                 return {"validator_name": "fake", "passed": True, "message": "ok"}
116
117         monkeypatch.setattr(wmain.validator_registry, "run_all_with_context",
118                             lambda path, cfg: ([_OK()], {}))
119         monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
120
121         cfg_file = tmp_path / "wcfg.json"
122         cfg_file.write_text(json.dumps({
123             "indexing": {"detector_impl": "stub", "skip_ai_handoff": True},
124             "output": {"output_dir": str(tmp_path / "wout")},
125         }))
126         video = tmp_path / "w.mp4"
127         video.write_bytes(b"FAKEVIDEO-PARITY")
128         out = tmp_path / "bucket"

```

```

129
130         rc = wmain.cmd_infer(wmain.build_parser().parse_args([
131             "infer", "--video-uri", str(video), "--out-uri", str(out), "--config",
str(cfg_file),
132             "--scratch", str(tmp_path / "scratch"), "--segmenter", "wasb_hybrid"]))
133         assert rc == 0
134
135         # Find the intervals.csv (outputs/<video_id>/intervals.csv).
136         intervals = next(out.rglob("intervals.csv"))
137         rows = intervals.read_text().splitlines()[1:] # drop header
138         worker = [(round(float(r.split(",")[0]), 3), round(float(r.split(",")[1]), 3)) for r
in rows]
139         assert worker == ref
140

```

8. user (2026-06-21T10:18:37.622Z)

```

1     """Per-profile startup checks (COMPUTE_DECOUPLED_SERVING M5).
2
3     L1 unit coverage of ``backend/config/startup.py``: the fail/warn-fast per-profile
checks.
4     Pure ? no torch/GPU/network (the CUDA probe + env are injected). The cardinal invariant:
5     **no profile selected ? ZERO checks** (default-OFF, identical to pre-M5).
6     """
7     import logging
8
9     import pytest
10
11     from backend.config import AppConfig
12     from backend.config.startup import (
13         LEVEL_ERROR,
14         LEVEL_WARN,
15         StartupCheckError,
16         enforce_startup_checks,
17         run_startup_checks,
18     )
19
20
21     def _cfg(profile="", backend="local", device="cuda", detector_impl="auto"):
22         """Build an AppConfig with arbitrary profile/storage/device combos ? including ones
a
23         preset would never produce (e.g. cloud-serving + local storage), to exercise the
coherence
24         checks. model_validate fills every other field with its default."""
25         return AppConfig.model_validate({
26             "deployment": {"profile": profile},
27             "storage": {"backend": backend},
28             "indexing": {"detector_impl": detector_impl, "wasb": {"device": device}},
29         })
30
31
32     def _codes(checks):
33         return {c.code for c in checks}
34
35
36     # ----- default-OFF
37
38     @pytest.mark.parametrize("backend,device,detector", [
39         ("local", "cuda", "auto"), ("gcs", "cpu", "stub"), ("s3", "auto", "native"),
40     ])
41     def test_no_profile_is_strict_noop(backend, device, detector):
42         """profile='' ? ZERO checks regardless of any other field (the pre-M5 behaviour)."""
43         cfg = _cfg(profile="", backend=backend, device=device, detector_impl=detector)
44         assert run_startup_checks(cfg, cuda_available=False, env={}) == []
45

```



```

46
47 def test_enforce_no_profile_never_raises_and_returns_empty():
48     cfg = _cfg(profile="", backend="local")
49     assert enforce_startup_checks(cfg, cuda_available=False, env={}) == []
50
51
52 # ----- truly-local
53
54 def test_truly_local_local_storage_gpu_present_is_clean():
55     cfg = _cfg(profile="truly-local", backend="local", device="cuda")
56     assert run_startup_checks(cfg, cuda_available=True, env={}) == []
57
58
59 def test_truly_local_cuda_strict_without_gpu_warns():
60     cfg = _cfg(profile="truly-local", backend="local", device="cuda")
61     checks = run_startup_checks(cfg, cuda_available=False, env={})
62     assert _codes(checks) == {"CUDA_NOT_VISIBLE"}
63     assert all(c.level == LEVEL_WARN for c in checks) # never fatal ? healthcheck is
the authority
64
65
66 def test_truly_local_auto_without_gpu_warns_cpu_fallback():
67     cfg = _cfg(profile="truly-local", backend="local", device="auto")
68     checks = run_startup_checks(cfg, cuda_available=False, env={})
69     assert _codes(checks) == {"CPU_FALLBACK"}
70
71
72 def test_truly_local_unprobed_cuda_skips_gpu_checks():
73     """cuda_available=None (not probed, e.g. the API server) ? no GPU heads-up at
all."""
74     cfg = _cfg(profile="truly-local", backend="local", device="cuda")
75     assert run_startup_checks(cfg, cuda_available=None, env={}) == []
76
77
78 def test_truly_local_cloud_storage_warns_unusual():
79     cfg = _cfg(profile="truly-local", backend="gcs", device="auto")
80     checks = run_startup_checks(cfg, cuda_available=True, env={})
81     assert "STORAGE_UNUSUAL" in _codes(checks)
82
83
84 def test_truly_local_gdrive_storage_is_fine():
85     cfg = _cfg(profile="truly-local", backend="gdrive", device="cuda")
86     assert run_startup_checks(cfg, cuda_available=True, env={}) == []
87
88
89 # ----- cloud-serving
90
91 def test_cloud_serving_local_storage_is_fatal():
92     cfg = _cfg(profile="cloud-serving", backend="local")
93     checks = run_startup_checks(cfg, cuda_available=True,
env={"GOOGLE_APPLICATION_CREDENTIALS": "/k"})
94     incoherent = [c for c in checks if c.code == "STORAGE_INCOHERENT"]
95     assert incoherent and incoherent[0].level == LEVEL_ERROR
96
97
98 def test_cloud_serving_gcs_without_creds_warns():
99     cfg = _cfg(profile="cloud-serving", backend="gcs", device="cuda")
100     checks = run_startup_checks(cfg, cuda_available=True, env={})
101     assert _codes(checks) == {"CREDS_UNCONFIRMED"}
102     assert all(c.level == LEVEL_WARN for c in checks)
103
104
105 def test_cloud_serving_gcs_with_creds_is_clean():
106     cfg = _cfg(profile="cloud-serving", backend="gcs", device="cuda")
107     assert run_startup_checks(cfg, cuda_available=True,

```

```

108         env={"GOOGLE_APPLICATION_CREDENTIALS": "/k"}) == []
109
110
111     def test_cloud_serving_on_cloud_run_env_skips_creds_warn():
112         """K_SERVICE (Cloud Run) implies ADC via the metadata server ? no creds warning."""
113         cfg = _cfg(profile="cloud-serving", backend="gcs", device="cuda")
114         assert run_startup_checks(cfg, cuda_available=True, env={"K_SERVICE": "rally"}) ==
115         []
116
117     def test_cloud_serving_cpu_device_warns():
118         cfg = _cfg(profile="cloud-serving", backend="gcs", device="cpu")
119         checks = run_startup_checks(cfg, cuda_available=True, env={"K_SERVICE": "rally"})
120         assert "CLOUD_CPU_DEVICE" in _codes(checks)
121
122
123     def test_cloud_serving_no_gpu_warns():
124         cfg = _cfg(profile="cloud-serving", backend="gcs", device="cuda")
125         checks = run_startup_checks(cfg, cuda_available=False, env={"K_SERVICE": "rally"})
126         assert "CLOUD_CUDA_NOT_VISIBLE" in _codes(checks)
127
128
129     # ----- cpu-mock
130
131     def test_cpu_mock_stub_is_clean():
132         cfg = _cfg(profile="cpu-mock", backend="local", detector_impl="stub")
133         assert run_startup_checks(cfg, cuda_available=False, env={}) == []
134
135
136     def test_cpu_mock_real_detector_warns():
137         cfg = _cfg(profile="cpu-mock", backend="local", detector_impl="native")
138         checks = run_startup_checks(cfg, cuda_available=False, env={})
139         assert "MOCK_NONSTUB_DETECTOR" in _codes(checks)
140
141
142     # ----- enforce
143
144     def test_enforce_raises_on_fatal_and_carries_failures(caplog):
145         cfg = _cfg(profile="cloud-serving", backend="local")
146         with caplog.at_level(logging.ERROR):
147             with pytest.raises(StartupCheckError) as ei:
148                 enforce_startup_checks(cfg, cuda_available=True, env={"K_SERVICE": "x"})
149         assert [c.code for c in ei.value.failures] == ["STORAGE_INCOHERENT"]
150         assert "STORAGE_INCOHERENT" in caplog.text # logged loudly at ERROR before raising
151
152
153     def test_enforce_does_not_raise_on_warnings_only():
154         cfg = _cfg(profile="truly-local", backend="local", device="auto")
155         checks = enforce_startup_checks(cfg, cuda_available=False, env={}) # CPU_FALLBACK
warning
156         assert checks and all(c.level == LEVEL_WARN for c in checks) # returned, not raised
157
158
159     def test_enforce_raise_on_error_false_returns_errors_without_raising():
160         cfg = _cfg(profile="cloud-serving", backend="local")
161         checks = enforce_startup_checks(cfg, cuda_available=True, env={"K_SERVICE": "x"},
162                                     raise_on_error=False)
163         assert any(c.code == "STORAGE_INCOHERENT" and c.level == LEVEL_ERROR for c in
checks)
164
165
166     # ----- worker wiring (fail fast before
any work)
167
168     def test_worker_cmd_infer_aborts_rc2_on_incoherent_profile(tmp_path):

```

```

169         """The worker's M5 hook aborts cleanly (rc 2) BEFORE any fetch/segmentation when the
active
170         profile is fatally incoherent. The cloud-serving preset sets storage=gcs;
config.json overrides
171         it back to local ? STORAGE_INCOHERENT ? fail fast."""
172         from backend.worker import __main__ as wmain
173
174         cfg = tmp_path / "config.json"
175         cfg.write_text('{"deployment": {"profile": "cloud-serving"}, "storage": {"backend":
"local"}}')
176         video = tmp_path / "in.mp4"
177         video.write_bytes(b"FAKE")
178         rc = wmain.cmd_infer(wmain.build_parser().parse_args([
179             "infer", "--video-uri", str(video), "--out-uri", str(tmp_path / "out"),
180             "--config", str(cfg), "--scratch", str(tmp_path / "s"), "--segmenter",
"wasb_hybrid"]))
181         assert rc == 2
182

```

9. assistant (2026-06-21T10:18:41.109Z)

Now let me read the remaining changed files and the key production code the tests exercise.

10. user (2026-06-21T10:18:42.129Z)

```

1         """Deployment-profile presets + environment auto-detection (COMPUTE_DECOUPLED_SERVING
M1).
2
3         A *deployment profile* is a single switch that applies a coherent BUNDLE of existing
4         config knobs, so the truly-local / cloud-serving / cpu-mock modes don't drift apart one
5         knob at a time (the design in ``docs/COMPUTE_DECOUPLED_SERVING/README.md`` §4.3).
6
7         Precedence (lowest ? highest), realised by the loader::
8
9         built-in model defaults < profile preset < config.json < env < worker --set
10
11         This module is **pure data + pure functions** ? no IO, and (critically) **no torch
import at
12         module load**. The loader applies a preset ONLY when a profile is EXPLICITLY selected
13         (``config.json`` ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); with no
14         profile selected the loader is byte-identical to the pre-M1 behaviour (``profile=""`` ==
15         today). Auto-detect only ever **suggests** a profile ? it never auto-applies one (D15:
cloud
16         spend is never entered silently).
17         """
18
19         from __future__ import annotations
20
21         import copy
22         import os
23         from typing import Any, Mapping, Optional
24
25         # Canonical enum values, defined ONCE here. ``models.DeploymentConfig``'s ``Literal``
mirrors
26         # these and ``tests/test_config_deployment.py`` pins parity so the two can never drift.
27         PROFILES: tuple[str, ...] = ("truly-local", "cloud-serving", "cpu-mock")
28         COMPUTE_TARGETS: tuple[str, ...] = ("local_gpu", "cloud_run", "cpu_mock")
29
30         # Each profile maps 1:1 to its canonical compute_target. Used to apply the preset and to
warn
31         # when config.json explicitly overrides compute_target into a state inconsistent with
the profile.
32         COMPUTE_TARGET_FOR_PROFILE: dict[str, str] = {
33             "truly-local": "local_gpu",
34             "cloud-serving": "cloud_run",

```

```

35     "cpu-mock": "cpu_mock",
36 }
37
38 # A preset is a partial, nested override dict over the model defaults. It touches ONLY
knobs
39 # that EXIST today ? M1 is pure/additive: the new
`deployment.{profile,compute_target}` plus
40 # `indexing.{detector_impl,skip_ai_handoff}` and `storage.backend`. Later milestones
extend
41 # these bundles ? `input_backend` (M2) and the configurable output/workspace base (M3,
42 # Launch-Report-gated) ? deliberately NOT set here so M1 never pre-empts a gated flip.
43 PROFILE_PRESETS: dict[str, dict[str, Any]] = {
44     "truly-local": {
45         "deployment": {"profile": "truly-local", "compute_target": "local_gpu"},
46         # detector_impl stays "auto" (the runner picks WSL vs native by env on the GPU
box).
47         # Zero-cloud / privacy-first ? run fully local with no Gemini handoff.
48         "indexing": {"skip_ai_handoff": True},
49         "storage": {"backend": "local"},
50     },
51     "cloud-serving": {
52         "deployment": {"profile": "cloud-serving", "compute_target": "cloud_run"},
53         # Linux L4 (cuda); Gemini handoff ON until the owned model clears the #172 floor
(D11).
54         "indexing": {"detector_impl": "native", "skip_ai_handoff": False},
55         "storage": {"backend": "gcs"},
56     },
57     "cpu-mock": {
58         "deployment": {"profile": "cpu-mock", "compute_target": "cpu_mock"},
59         # Deterministic CPU mock ? no GPU/WSL, no provider/API. The CI / regression
profile.
60         "indexing": {"detector_impl": "stub", "skip_ai_handoff": True},
61         "storage": {"backend": "local"},
62     },
63 }
64
65
66 def is_known_profile(profile: str) -> bool:
67     """True for a non-empty, recognised profile name."""
68     return bool(profile) and profile in PROFILE_PRESETS
69
70
71 def preset_for(profile: str) -> dict[str, Any]:
72     """Return a deep COPY of the preset bundle for `profile` (so callers can't mutate
the
73     module-level table), or `{}` for `` / an unknown profile."""
74     return copy.deepcopy(PROFILE_PRESETS.get(profile, {}))
75
76
77 def _truthy(val: Optional[str]) -> bool:
78     """Common env-var truthiness (`1/true/yes/on`, case/space-insensitive)."""
79     if val is None:
80         return False
81     return val.strip().lower() in {"1", "true", "yes", "on"}
82
83
84 def _cuda_available() -> bool:
85     """Best-effort CUDA probe. torch is heavy + optional, so it is imported lazily here
and
86     ONLY when a caller opts into `probe_cuda`; any import/runtime failure means 'no
GPU'."""
87     try:
88         # Intentionally lazy + local: keeps the config-load path torch-free (torch is
heavy
89         # and optional). Only reached when a caller opts into probe_cuda.

```

```

90         import torch
91
92         return bool(torch.cuda.is_available())
93     except Exception:
94         return False
95
96
97     def probe_cuda_available() -> bool:
98         """Public best-effort CUDA probe (lazy torch import; ``False`` on any failure).
99
100         Exposed for the per-profile startup checks (``backend/config/startup.py``) so a
101         caller
102         that *wants* the GPU heads-up can pass the result in without the startup module
103         importing
104         torch itself. Best-effort by design: if torch lives only in a detector subprocess
105         env
106         (native WASB), this returns ``False`` in the main process ? which is why the startup
107         checks
108         treat a GPU mismatch as a WARNING (the runner healthcheck is the authority), never a
109         hard
110         error that could false-fail."""
111         return _cuda_available()
112
113     def suggest_profile(
114         env: Optional[Mapping[str, str]] = None, *, probe_cuda: bool = False
115     ) -> str:
116         """SUGGEST (never auto-apply ? D15) a deployment profile from the environment.
117
118         Signals, in priority order:
119
120         - ``K_SERVICE`` set (Cloud Run)           ? ``"cloud-serving"``
121         - ``CI`` truthy                            ? ``"cpu-mock"``
122         - ``probe_cuda`` and a CUDA GPU present ? ``"truly-local"``
123         - otherwise                               ? ``""`` (no strong suggestion)
124
125         ``probe_cuda`` defaults to ``False`` so the common config-load path stays cheap and
126         never
127         imports torch; a CLI ``--detect`` or cloud-serving startup that wants the GPU probe
128         passes
129         ``probe_cuda=True``. The result is advisory only: the caller decides whether to act
130         on it.
131
132         """
133         e = os.environ if env is None else env
134         if e.get("K_SERVICE"):
135             return "cloud-serving"
136         if _truthy(e.get("CI")):
137             return "cpu-mock"
138         if probe_cuda and _cuda_available():
139             return "truly-local"
140         return ""

```

11. user (2026-06-21T10:18:42.557Z)

```

1     """Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.
2
3     python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
4         [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
5     python -m backend.worker train ...          # Gen-0 train-from-bucket (follow-up)
6
7     The URI scheme selects the storage backend via ``backend.storage`` (a bare path /
8     ``local://``
9     is an identity passthrough; ``s3://`` / ``gcs://`` is an object store), so existing
10    local runs

```

```

9         are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
10         ``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini
key) ?
11         the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).
12
13         This is a thin orchestrator: it composes the SAME building blocks the ``backend.main``
CLI uses
14         (validator registry ? segmenter ? report), never a forked pipeline.
15         """
16         from __future__ import annotations
17
18         import argparse
19         import csv
20         import json
21         import logging
22         import os
23         import sys
24         import tempfile
25         from typing import Any, List
26
27         from backend import storage
28         from backend.config import (
29             StartupCheckError,
30             enforce_startup_checks,
31             load_config,
32             probe_cuda_available,
33         )
34         from backend.infrastructure.database import Database
35         from backend.pipeline.segmenters.base import segmenter_registry
36         from backend.pipeline.validators.base import registry as validator_registry
37         from backend.reporting import emit_indexer_report
38         from backend.results import Failure
39         from backend.utils.hashing import compute_video_id
40
41         logger = logging.getLogger("backend.worker")
42
43
44         def _coerce(v: str) -> Any:
45             """Coerce a ``--set`` string value to bool/int/float where it round-trips, else
str."""
46             low = v.lower()
47             if low in ("true", "false"):
48                 return low == "true"
49             for cast in (int, float):
50                 try:
51                     return cast(v)
52                 except ValueError:
53                     pass
54             return v
55
56
57         def _apply_overrides(config: Any, sets: List[str]) -> None:
58             """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
indexing.skip_ai_handoff=true)."""
59             for kv in sets or []:
60                 key, sep, val = kv.partition("=")
61                 if not sep:
62                     raise ValueError(f"--set expects k=v, got {kv!r}")
63                 parts = key.split(".")
64                 obj = config
65                 for p in parts[:-1]:
66                     obj = getattr(obj, p)
67                 setattr(obj, parts[-1], _coerce(val))
68
69

```

```

70     def _write_intervals_csv(segments: List[dict], path: str) -> None:
71         """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-
friendly
72         artifact (same schema as the rally-annotator labels)."""
73         with open(path, "w", newline="") as f:
74             w = csv.writer(f)
75             w.writerow(["start", "end", "shot_count", "ending_reason"])
76             for s in segments:
77                 w.writerow([
78                     f'{float(s.get("start_time", 0.0)):.3f}',
79                     f'{float(s.get("end_time", 0.0)):.3f}',
80                     s.get("shot_count", ""),
81                     s.get("ending_reason", s.get("shot_count_confidence", "")),
82                 ])
83
84
85     def _write_report_json(video_id: str, segments: List[dict], status: str, path: str) ->
None:
86         with open(path, "w") as f:
87             json.dump({
88                 "video_id": video_id,
89                 "status": status,
90                 "segment_count": len(segments),
91                 "segments": [{"start": float(s.get("start_time", 0.0)),
92                             "end": float(s.get("end_time", 0.0))} for s in segments],
93             }, f, indent=2)
94
95
96     def _run_startup_checks(config) -> bool:
97         """M5 per-profile startup checks for the worker (the compute process, so it probes
CUDA
98         best-effort). Logs warnings; returns False on a FATAL coherence error so the caller
aborts
99         cleanly (rc 2) rather than run an expensive job under an incoherent profile. A no-op
(return
100         True) when no deployment.profile is selected."""
101         try:
102             enforce_startup_checks(config, cuda_available=probe_cuda_available(),
logger=logger)
103             return True
104         except StartupCheckError:
105             return False # already logged at ERROR by enforce_startup_checks
106
107
108     def cmd_infer(args: argparse.Namespace) -> int:
109         config = load_config(args.config) if args.config else load_config()
110         _apply_overrides(config, args.set)
111         if not _run_startup_checks(config):
112             return 2
113         storage_cfg = config.storage.model_dump()
114
115         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
116         os.makedirs(scratch, exist_ok=True)
117         config.output.output_dir = scratch
118
119         # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
to scratch.
120         local_video = storage.fetch(args.video_uri, os.path.join(scratch, "in.mp4"),
config=storage_cfg)
121         print(f"[worker] pulled {args.video_uri} -> {local_video}")
122
123         # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
byte-identical).
124         video_id = compute_video_id(local_video)
125

```

```

126     # 3. VALIDATE (same registry as the main CLI).
127     val_results, val_context = validator_registry.run_all_with_context(local_video,
config)
128     failures = [r for r in val_results if not r.passed]
129     db = Database(os.path.join(scratch, config.output.db_name))
130     db.add_video(video_id, local_video, "failed" if failures else "processed",
131                 [r.model_dump() for r in val_results])
132
133     segments: List[dict] = []
134     segmenter_actual = None
135     segmentation_ran = False
136     status = "failed"
137     try:
138         if failures:
139             print("[worker] validation FAILED: "
140                   + "; ".join(f"{r.validator_name}: {r.message}" for r in failures))
141             return 2
142         seg_name = args.segmenter or config.indexing.default_segmenter
143         segmenter_cls = segmenter_registry.get(seg_name)
144         if not segmenter_cls:
145             print(f"[worker] unknown segmenter: {seg_name!r} "
146                   f"(have {list(segmenter_registry.list_available())})")
147             return 2
148         segmenter_actual = seg_name
149         result = segmenter_cls(db, config).process_video(video_id, local_video,
max_frames=args.max_frames)
150         segmentation_ran = True
151         if isinstance(result, Failure):
152             print(f"[worker] segmenter FAILED: {result.message}")
153             return 3
154         segments = result.value if hasattr(result, "value") else result
155         status = "success"
156         # Loud, never-silent diagnostic: a 0-segment success is almost always a
misconfig, not an
157         # empty match ? the cold-start failure mode (the substrate detector yielded no
usable
158         # trajectories). Explain it so a run is analysable from the log alone (re-run -v
for more).
159         if not segments:
160             detector_impl = (os.environ.get("RALLY_DETECTOR_IMPL")
or getattr(config.indexing, "detector_impl", None) or
"auto")
161             logger.warning(
162                 "infer produced 0 segments for video_id=%s (segmenter=%s,
detector_impl=%s). "
163                 "The detector substrate yielded no usable trajectories/windows ? check
the detector "
164                 "logs above; common causes: native runner UNAVAILABLE
(weights/repo/WASB_PYTHON env), "
165                 "the WASB env produced no detections, or the input resolution differs
from the tuned "
166                 "proxy resolution. Re-run with -v for the native command + frame
counts.",
167                 video_id, segmenter_actual, detector_impl,
168             )
169         finally:
170             # Best-effort per-video observability report (never raises) ? parity with the
main CLI.
171             emit_indexer_report(
172                 db=db, video_id=video_id, video_path=local_video, config=config,
173                 val_results=val_results, val_context=val_context,
174                 segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
175                 was_reingest=False, segmentation_ran=segmentation_ran,
176             )
177
178

```



```

179     # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
180     out_local = os.path.join(scratch, "outputs", video_id)
181     os.makedirs(out_local, exist_ok=True)
182     _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
183     _write_report_json(video_id, segments, status, os.path.join(out_local,
"report.json"))
184
185     out_prefix = args.out_uri.rstrip("/")
186     pushed = []
187     for fn in sorted(os.listdir(out_local)):
188         uri = f"{out_prefix}/outputs/{video_id}/{fn}"
189         storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
190         pushed.append(uri)
191
192     print(f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):")
193     for u in pushed:
194         print("    ", u)
195     return 0
196
197
198     #: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter
so a
199     #: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem
collision).
200     _VIDEO_EXTS = (".mp4", ".mov", ".mkv", ".avi", ".webm", ".m4v")
201
202
203     def _probe_video_fps_width(path: str):
204         """Robust ffprobe (fps, frame_width) for a real video. Returns None on ANY failure.
205
206         The detector trajectory is frame-indexed; the harness maps frame->time via fps, so a
WRONG
207         fps silently mis-aligns every rally label (e.g. a 60fps clip read as 30 doubles
every time).
208         cv2's ``_probe_fps_width`` silently defaults to (30, 1280) on a read miss ? fine for
the
209         inference report (it flags the fallback) but corrupting for REAL training data. So
here we
210         probe with ffprobe (same tool as ``get_video_duration``) and return None so the
caller SKIPS
211         rather than guesses.
212         """
213         import json
214         import subprocess
215         try:
216             out = subprocess.check_output(
217                 ["ffprobe", "-v", "error", "-select_streams", "v:0",
218                  "-show_entries", "stream=r_frame_rate,width", "-of", "json", path],
219                 stderr=subprocess.DEVNULL).decode()
220             st = (json.loads(out).get("streams") or [{}])[0]
221             num, _, den = str(st.get("r_frame_rate", "0/1")).partition("/")
222             fps = float(num) / float(den or "1")
223             width = float(st.get("width") or 0)
224             if fps > 0 and width > 0:
225                 return fps, width
226             except (subprocess.SubprocessError, ValueError, KeyError, OSError,
json.JSONDecodeError):
227                 pass
228             return None
229
230
231     def _resolve_train_detector(args: argparse.Namespace, config: Any):
232         """Build the trajectory DetectorRunner for ``--trajectory-source detector``, honouring
233         detector_impl (RALLY_DETECTOR_IMPL / indexing.detector_impl: auto=WSL, native=GPU

```

```

box,
234 stub=CPU mock). Returns the runner, or prints an error + returns None.
235
236 Reuses the segmenter's `_resolve_runner` seam so the worker and the live CLI build
the
237 detector identically (one source of truth). A non-trajectory segmenter (e.g. yolo)
has
238 no shuttle detector and is rejected.
239 """
240 from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
241
242 seg_cls = segmenter_registry.get(args.segmenter)
243 if not seg_cls:
244     print(f"[worker] unknown segmenter: {args.segmenter!r} "
245           f"(have {list(segmenter_registry.list_available())})")
246     return None
247 seg = seg_cls(None, config)
248 if not isinstance(seg, TrajectoryHybridSegmenter):
249     print(f"[worker] --segmenter {args.segmenter!r} has no shuttle detector; "
250           f"use a trajectory hybrid (wasb_hybrid / tracknet_hybrid /
fusion_hybrid).")
251     return None
252 try:
253     runner, _ = seg._resolve_runner(config.get("indexing", {}))
254 except NotImplementedError as e:
255     # e.g. detector_impl='native' for a family without a native runner (tracknet, or
256     # fusion with a tracknet substrate). Fail cleanly (rc!=0), not with a raw
traceback.
257     print(f"[worker] detector not available: {e}")
258     return None
259 ok, msg = runner.healthcheck()
260 if not ok:
261     print(f"[worker] detector environment not ready: {msg}")
262     return None
263 print(f"[worker] detector ready ({args.segmenter}): {msg}")
264 return runner
265
266
267 def cmd_train(args: argparse.Namespace) -> int:
268     """Gen-0 train-from-bucket: pull labels (+ videos) -> trajectories -> manifest ->
shell out
269 to training.gen0.harness (backend must NOT import training) -> push the artifact
bundle.
270
271 Two trajectory sources (the train pipeline + harness are identical either way):
272 * ``synth`` (default) ? the CPU "mock GPU": deterministic, label-consistent
synthetic
273 shuttle paths (no GPU/WSL), so the whole pipeline is validated on a CPU box /
CI.
274 * ``detector`` ? REAL shuttle trajectories: pull each video and run the resolved
275 DetectorRunner (native WASB on a GPU box, or stub for a GPU-free wiring test).
This
276 is the M3.5 "first real training" path; only the trajectory source flips.
277 """
278 import subprocess
279
280 config = load_config(args.config) if args.config else load_config()
281 _apply_overrides(config, args.set)
282 if not _run_startup_checks(config):
283     return 2
284 storage_cfg = config.storage.model_dump()
285
286 scratch = args.scratch or tempfile.mkdtemp(prefix="rally-train-")
287 labels_dir = os.path.join(scratch, "labels")
288 traj_dir = os.path.join(scratch, "trajectories")

```

```

289     videos_dir = os.path.join(scratch, "videos")
290     os.makedirs(labels_dir, exist_ok=True)
291     os.makedirs(traj_dir, exist_ok=True)
292
293     corpus = args.corpus_uri.rstrip("/")
294     label_uris = sorted(u for u in storage.list_uris(f"{corpus}/labels/",
config=storage_cfg)
295                         if u.lower().endswith(".csv"))
296     if len(label_uris) < 2:
297         print(f"[worker] need >=2 labeled videos for leave-one-video-out; found
{len(label_uris)} "
298             f"under {corpus}/labels/")
299     return 2
300
301     # Real-detector source: resolve the runner once + index the bucket's videos/ by
stem.
302     runner = None
303     video_by_stem: dict = {}
304     if args.trajectory_source == "detector":
305         os.makedirs(videos_dir, exist_ok=True)
306         runner = _resolve_train_detector(args, config)
307         if runner is None:
308             return 2
309         for vuri in storage.list_uris(f"{corpus}/videos/", config=storage_cfg):
310             vname = storage.parse_uri(vuri).name
311             if not vname.lower().endswith(_VIDEO_EXTS):
312                 continue # skip sidecars (.srt/.json/thumbnails) so they can't shadow
the video
313             video_by_stem[os.path.splitext(vname)[0]] = vuri
314
315         print(f"[worker] trajectory source: {args.trajectory_source}")
316         specs: List[dict] = []
317         for uri in label_uris:
318             fname = storage.parse_uri(uri).name # e.g.
GX020094_proxy.rallies.csv
319             name = fname.replace(".rallies.csv", "").replace(".csv", "")
320             local_label = storage.fetch(uri, os.path.join(labels_dir, fname),
config=storage_cfg)
321             if args.trajectory_source == "detector":
322                 vuri = video_by_stem.get(name)
323                 if not vuri:
324                     print(f"[worker] no video for {name!r} under {corpus}/videos/ ?
skipping.")
325                     continue
326                     local_video = storage.fetch(vuri, os.path.join(videos_dir,
storage.parse_uri(vuri).name),
327                                             config=storage_cfg)
328                     video_id = compute_video_id(local_video)
329                     # Real fps/width drive the trajectory frame->time mapping; PROBE (don't
guess) ? a
330                     # wrong fps silently mis-aligns every label, so skip the video if ffprobe
can't read it.
331                     meta = _probe_video_fps_width(local_video)
332                     if meta is None:
333                         print(f"[worker] could not probe fps/width for {name!r} (ffprobe) ?
skipping (won't guess).")
334                         continue
335                     fps, frame_width = meta
336                     traj = runner.run_predict(local_video, traj_dir, video_id=video_id)
337                     if not traj:
338                         print(f"[worker] detector produced no trajectory for {name!r} ?
skipping.")
339                         continue
340                     specs.append({"name": name, "trajectory": traj, "labels": local_label,
"fps": fps, "frame_width": frame_width})
341

```

```

342         else:
343             from backend.pipeline.detectors.stub_runner import
synth_trajectory_from_labels
344             traj = os.path.join(traj_dir, f"{name}_traj.csv")
345             synth_trajectory_from_labels(local_label, traj, fps=args.fps,
frame_width=args.frame_width)
346             specs.append({"name": name, "trajectory": traj, "labels": local_label,
347                           "fps": args.fps, "frame_width": args.frame_width})
348
349         if len(specs) < 2:
350             print(f"[worker] need >=2 videos with trajectories for leave-one-video-out; got
{len(specs)}.")
351             return 2
352         manifest_path = os.path.join(scratch, "manifest.json")
353         with open(manifest_path, "w", encoding="utf-8") as f:
354             json.dump(specs, f, indent=2)
355         src_label = "real detector" if args.trajectory_source == "detector" else "CPU-mock
stub"
356         print(f"[worker] built manifest: {len(specs)} videos ({src_label} trajectories)")
357
358         out_dir = os.path.join(scratch, "artifacts")
359         cmd = [sys.executable, "-m", "training.gen0.harness",
360               "--manifest", manifest_path, "--out", out_dir, "--version", args.version]
361         if args.floor:
362             floor_local = (storage.fetch(args.floor, os.path.join(scratch, "floor.json"),
config=storage_cfg)
363                           if "://" in args.floor else args.floor)
364             cmd += ["--floor", floor_local]
365         print("[worker] training:", " ".join(cmd))
366         rc = subprocess.run(cmd).returncode
367         if rc != 0:
368             print(f"[worker] gen0 harness failed (rc={rc})")
369             return rc
370
371         # PUSH the versioned artifact bundle (weights.json + manifest.json) to the bucket.
372         bundle = os.path.join(out_dir, args.version)
373         out_prefix = args.out_uri.rstrip("/")
374         pushed = []
375         for fn in sorted(os.listdir(bundle)):
376             uri = f"{out_prefix}/weights/{args.version}/{fn}"
377             storage.put(os.path.join(bundle, fn), uri, config=storage_cfg)
378             pushed.append(uri)
379         print(f"[worker] train done: pushed {len(pushed)} artifact(s):")
380         for u in pushed:
381             print("    ", u)
382         return 0
383
384
385     def build_parser() -> argparse.ArgumentParser:
386         p = argparse.ArgumentParser(prog="python -m backend.worker",
387                                     description="Storage-aware worker: pull ? run pipeline ?
push.")
388         p.add_argument("-v", "--verbose", action="store_true",
389                       help="DEBUG logging (the native detector command, frame counts, per-
stage detail)")
390         sub = p.add_subparsers(dest="cmd", required=True)
391
392         pi = sub.add_parser("infer", help="run inference on one video URI and push outputs")
393         pi.add_argument("--video-uri", required=True, help="local path / local:// / s3:// /
gcs://")
394         pi.add_argument("--out-uri", required=True, help="output prefix
(outputs/<video_id>/? is appended)")
395         pi.add_argument("--segmenter", default=None, help="default:
indexing.default_segmenter")
396         pi.add_argument("--config", default=None, help="config.json path (default:

```

```

./config.json)")
397     pi.add_argument("--scratch", default=None, help="local scratch dir (default: a temp
dir)")
398     pi.add_argument("--max-frames", type=int, default=None)
399     pi.add_argument("--set", action="append", default=[], metavar="k=v",
400                     help="config override, e.g. --set indexing.skip_ai_handoff=true")
401     pi.set_defaults(func=cmd_infer)
402
403     pt = sub.add_parser("train", help="Gen-0 train-from-bucket (labels [+videos] ->
trajectories -> harness)")
404     pt.add_argument("--corpus-uri", required=True, help="prefix containing labels/ (+
videos/ for "
405                     "--trajectory-source detector), e.g. s3://bucket or gcs://bucket")
406     pt.add_argument("--out-uri", required=True, help="output prefix (weights/<version>/
is appended)")
407     pt.add_argument("--version", required=True, help="semver for the artifact bundle")
408     pt.add_argument("--trajectory-source", choices=["synth", "detector"],
default="synth",
409                     help="synth = CPU-mock label-consistent trajectories (default, no
GPU); "
410                     "detector = REAL shuttle trajectories via the resolved
DetectorRunner "
411                     "(needs videos/ in the corpus; honours detector_impl).")
412     pt.add_argument("--segmenter", default="wasb_hybrid",
413                     help="trajectory hybrid whose detector is used for --trajectory-
source detector")
414     pt.add_argument("--floor", default=None, help="floor baseline JSON (local path or
storage URI)")
415     pt.add_argument("--config", default=None)
416     pt.add_argument("--scratch", default=None)
417     pt.add_argument("--fps", type=float, default=30.0, help="synth trajectory fps
(detector probes the video)")
418     pt.add_argument("--frame-width", type=float, default=1280.0,
419                     help="synth trajectory frame width (detector probes the video)")
420     pt.add_argument("--set", action="append", default=[], metavar="k=v")
421     pt.set_defaults(func=cmd_train)
422     return p
423
424
425     def _setup_logging(verbose: bool) -> None:
426         """Configure logging for the worker CLI so the pipeline's existing INFO/ERROR
diagnostics are
427         VISIBLE ? the worker previously ran with no logging config, so every
segmenter/detector log
428         (detector choice, 'Running native WASB inference', availability failures) was
silently dropped.
429         That is exactly why a 0-rally cold-start gave nothing to analyse. Configured here
(the CLI
430         entry), not at import, so importing the module never reconfigures a caller's
logging."""
431         # force=True: basicConfig is a no-op if the root already has handlers (e.g. a
library
432         # configured logging at import), which would leave the level at WARNING and re-
suppress the
433         # diagnostics. Forcing reconfiguration means the worker reliably controls its own
verbosity.
434         logging.basicConfig(
435             level=logging.DEBUG if verbose else logging.INFO,
436             format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
437             datefmt="%H:%M:%S",
438             force=True,
439         )
440
441
442     def main(argv=None) -> int:

```

```

443     args = build_parser().parse_args(argv)
444     _setup_logging(getattr(args, "verbose", False))
445     return args.func(args)
446
447
448 if __name__ == "__main__":
449     sys.exit(main())
450

```

12. user (2026-06-21T10:18:43.484Z)

```

1     import os
2     import sys
3     import argparse
4     import logging
5     import uuid
6     from urllib.parse import quote
7     import uvicorn
8     from dotenv import load_dotenv
9
10    load_dotenv()
11
12    # Centralized logging setup for the indexer (hot paths use logger, CLI output uses print
for direct user-facing messages).
13    logging.basicConfig(
14        level=logging.INFO,
15        format='%(asctime)s [%(levelname)s] %(name)s: %(message)s',
16        datefmt='%H:%M:%S'
17    )
18    logger = logging.getLogger(__name__)
19    from fastapi import FastAPI, HTTPException
20    from fastapi.middleware.cors import CORSMiddleware
21    from pydantic import BaseModel
22    from typing import List, Dict, Any, Optional
23
24    from fastapi.staticfiles import StaticFiles
25    from fastapi.responses import FileResponse, HTMLResponse, JSONResponse
26    from backend.pipeline.validators.base import registry
27    from backend.infrastructure.database import Database
28    from backend.pipeline.segmenters.base import segmenter_registry
29    from backend.pipeline.stitcher.compiler import VideoCompiler
30    from backend.utils.hashing import compute_video_id # canonical file-identity hashing
31    from backend.utils.paths import resolve_under_root, safe_output_filename,
validate_input_path
32    from backend.config import ( # new typed config
33        load_config as load_app_config,
34        AppConfig,
35        StitchingConfig,
36        enforce_startup_checks,
37        StartupCheckError,
38    )
39    from backend.results import Failure # standardized error/result contract
40    from backend.reporting import emit_indexer_report
41    from backend import storage # M2: URI-aware input acquisition (local = identity
passthrough)
42
43    app = FastAPI(title="Sports Highlight Indexer API")
44
45    # Enable CORS for the local web interface. allow_origins='*' with allow_credentials=True
is
46    # rejected by browsers per the fetch spec and is an unsafe default to carry into the
47    # auth/tenancy work; this tool uses no cookies, so credentials stay off.
48    app.add_middleware(
49        CORSMiddleware,
50        allow_origins=["*"],

```

```

51         allow_credentials=False,
52         allow_methods=["*"],
53         allow_headers=["*"],
54     )
55
56     # Serves static frontend files directly (now under api/ per approved structure)
57     os.makedirs("backend/api/static", exist_ok=True)
58     app.mount("/static", StaticFiles(directory="backend/api/static"), name="static")
59
60     @app.get("/", response_class=HTMLResponse)
61     def serve_index():
62         index_path = "backend/api/static/index.html"
63         if os.path.exists(index_path):
64             with open(index_path, "r", encoding="utf-8") as f:
65                 return f.read()
66         return "<h3>Sports Highlight Indexer Server is Running. Please create index.html in
backend/api/static/</h3>"
67
68     # Global variables initialized at startup
69     db_instance: Optional[Database] = None
70     global_config: Optional[AppConfig] = None # now typed (Pydantic model)
71
72     # Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
P3)
73     # now lives in backend.api.job_worker. Re-exported here so the names stay addressable on
74     # `backend.main` (`JobWaiting`, `_drain_due_jobs`, `_arm_wake_timer`, `_get_executor`,
75     # `_run_claimed_job`, `_MAX_DRAIN_WAKE_SEC`, ...). The worker resolves these seams
THROUGH
76     # this module object at call-time, so `monkeypatch.setattr(backend.main, ...)` still
77     # intercepts the inter-function calls (a parked job arms the PATCHED `_arm_wake_timer`,
78     # so no real daemon timer leaks in tests). Behavior is unchanged by the extraction.
79     from backend.api.job_worker import ( # noqa: F401 (re-exports: names stay addressable
on backend.main)
80         JobWaiting,
81         _arm_wake_timer,
82         _drain_due_jobs,
83         _get_executor,
84         _job_to_request,
85         _MAX_DRAIN_WAKE_SEC,
86         _run_claimed_job,
87         _schedule_next_drain_if_waiting,
88     )
89
90     # Legacy thin wrapper for any remaining direct calls during transition.
91     # New code should import load_app_config / AppConfig from backend.config directly.
92     def load_config(config_path: str = "config.json") -> Dict[str, Any]:
93         cfg = load_app_config(config_path)
94         return cfg.model_dump(mode="json")
95
96     # --- API Models ---
97     class ProcessRequest(BaseModel):
98         video_path: str
99         player_pool: Optional[List[str]] = None
100         max_frames: Optional[int] = None
101         segmenter: Optional[str] = None
102
103     class QueryRequest(BaseModel):
104         video_id: str
105         server: Optional[str] = None
106         receiver: Optional[str] = None
107         duration_min: Optional[float] = None
108         event_type: Optional[str] = None
109
110     class CompileRequest(BaseModel):
111         video_id: str

```

```

112     segment_ids: List[int]
113     output_filename: str
114
115 def _finalize_reingest(video_id: str, segments, play_wm: int, cand_wm: int) -> None:
116     """Commit a (re)segmentation with destroy-AFTER-confirm semantics.
117
118     On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
119     fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark)
120     and
121     keep the prior good results intact ? a failed/empty re-run never destroys prior
122     data.
123     """
124     if segments:
125         db_instance.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
126     else:
127         db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
128
129 # --- API Endpoints ---
130 @app.get("/api/config")
131 def get_config():
132     return global_config
133
134 @app.get("/api/health")
135 def health():
136     """Cheap liveness/readiness probe (no auth) for the SPA's offline banner
137     (HOSTING_PLAN.md:42). Reports the configured sport + default segmenter so the UI can
138     confirm the engine is reachable and which detector is wired. Never raises."""
139     sport = getattr(global_config, "sport", None)
140     indexing = getattr(global_config, "indexing", None)
141     return {
142         "status": "ok",
143         "sport": sport,
144         "default_segmenter": getattr(indexing, "default_segmenter", None),
145     }
146
147 def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
148     """The full hash ? validate ? segment ? report pipeline for one video.
149
150     This is the former synchronous /api/process body, unchanged in behavior, now run on
151     the job worker thread (DEFERRED_HARDENING_PLAN #16). Returns the same response dict
152     the sync endpoint used to return (UI compatibility); the per-video observability
153     report is still emitted in `finally:` and grafted as response["reports"].
154
155     `progress` is an optional callable(stage: str) used to surface coarse job progress.
156     Raises on unexpected internal errors ? the caller records those as a job failure
157     (after this function has already restored the pre-run segments, see below).
158     """
159     notify = progress or (lambda stage: None)
160
161     notify("hashing")
162     # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
163     (identity ? # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
164     original # req.video_path (the source URI) is kept for the DB record + report; only the file-
165     reading # consumers (hash / validators / segmenter) use the resolved local path.
166     local_video = storage.acquire_local_input(req.video_path, getattr(global_config,
167     "storage", None))
168     video_id = compute_video_id(local_video)
169     was_reingest = db_instance.get_video(video_id) is not None
170
171     # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
172     report)

```



```

170     notify("validating")
171     logger.info(f"Running validations for {req.video_path}...")
172     val_results, val_context = registry.run_all_with_context(local_video, global_config)
173     failures = [r for r in val_results if not r.passed]
174
175     # typed AppConfig: attribute access for the default segmenter (with safe fallback)
176     default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
"motion")
177     seg_name = req.segmenter or default_seg
178     segmenter_actual = None
179     segmentation_ran = False
180     response: Optional[Dict[str, Any]] = None
181     try:
182         if failures:
183             # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
184             # status; it no longer destroys the video's prior good segments.
185             db_instance.add_video(video_id, req.video_path, "failed", [r.model_dump()
for r in val_results])
186             response = {
187                 "video_id": video_id,
188                 "status": "failed",
189                 "message": "Video failed validation checks.",
190                 "results": [r.model_dump() for r in val_results],
191             }
192             return response
193
194         # 2. Run segmenter & indexer
195         logger.info("[API] Video passed checks. Segmenting and indexing...")
196         db_instance.add_video(video_id, req.video_path, "processed", [r.model_dump() for
r in val_results])
197
198         segmenter_cls = segmenter_registry.get(seg_name)
199         if not segmenter_cls:
200             # Submit-time validation already 400s unknown names; this is the defensive
201             # in-worker path (e.g. config default pointing at an unregistered
segmenter).
202             available = list(segmenter_registry.list_available().keys())
203             response = {
204                 "video_id": video_id,
205                 "status": "failed",
206                 "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
207                 "results": [r.model_dump() for r in val_results],
208                 "play_segments": [],
209             }
210             return response
211
212         logger.info(f"[API] Using segmenter: {seg_name}")
213         notify(f"segmenting ({seg_name})")
214         segmenter_actual = seg_name
215         # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
216         # if the run yields segments; otherwise drop the new partial rows and keep the
old.
217         play_wm, cand_wm = db_instance.segment_watermarks(video_id)
218         segmenter = segmenter_cls(db_instance, global_config)
219         try:
220             result = segmenter.process_video(video_id, local_video, req.player_pool,
req.max_frames)
221         except Exception:
222             # A raising segmenter must not leave half-written rows: restore the pre-run
223             # state (same destroy-after-confirm contract as the Failure branch), then
let
224             # the job worker record the failure.
225             db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)

```

```

226         raise
227     segmentation_ran = True
228
229     if isinstance(result, Failure):
230         logger.error(f"[API] Segmenter failed: {result.message}")
231         db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
232         response = {
233             "video_id": video_id,
234             "status": "failed",
235             "message": f"Segmenter '{seg_name}' failed: {result.message}",
236             "results": [r.model_dump() for r in val_results],
237             "play_segments": [],
238         }
239         return response
240
241     segments = result.value if hasattr(result, "value") else result
242     notify("finalizing")
243     _finalize_reingest(video_id, segments, play_wm, cand_wm)
244
245     response = {
246         "video_id": video_id,
247         "status": "success",
248         "message": f"Successfully indexed {len(segments)} play segments using
249 '{seg_name}' segmenter.",
250         "results": [r.model_dump() for r in val_results],
251         "play_segments": segments,
252     }
253     return response
254
255     finally:
256         # Always emit the per-video observability report (next to the video, or to
257         # report.output_dir). Never raises. Surface the paths in the response.
258         paths = emit_indexer_report(
259             db=db_instance, video_id=video_id, video_path=req.video_path,
260             config=global_config,
261             val_results=val_results, val_context=val_context,
262             segmenter_requested=req.segmenter, segmenter_actual=segmenter_actual,
263             was_reingest=was_reingest, segmentation_ran=segmentation_ran,
264         )
265         if paths and isinstance(response, dict):
266             response["reports"] = paths
267
268 @app.post("/api/process", status_code=202)
269 def process_video(req: ProcessRequest):
270     """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.
271
272     Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
273     client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
274     runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
275     worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
276     """
277     allowed_root = getattr(getattr(global_config, "security", None),
278 "allowed_video_root", None)
279     try:
280         validate_input_path(req.video_path, allowed_root=allowed_root)
281         # M2: resolve the input once (path or storage URI) ? raises ValueError for an
282         unsupported
283         # scheme, which we map to a clean 400 (an unknown scheme must never 500).
284         input_ref = storage.resolve_input_ref(req.video_path, getattr(global_config,
285 "storage", None))
286     except ValueError as e:
287         raise HTTPException(status_code=400, detail=str(e)) from e
288     # The local-existence fast-fail applies only to a LOCAL input, and stats the
289     RESOLVED local path
290     # (input_ref.key ? e.g. local://? stripped to the bare path), not the raw URI

```

```

string. A
285         # bucket/Drive URI can't be cheaply os.path.exists()'d ? its existence is verified
when the worker
286         # fetches it (a fetch miss fails the job loudly). validate_input_path above still
runs for every
287         # input: its null-byte guard always applies, and a configured allowed_video_root
(hosted mode)
288         # rejects a non-local URI as out-of-root.
289         if input_ref.backend == "local" and not os.path.exists(input_ref.key):
290             raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}'")
291         if req.segmenter and not segmenter_registry.get(req.segmenter):
292             available = list(segmenter_registry.list_available().keys())
293             raise HTTPException(
294                 status_code=400,
295                 detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
{available}")
296         )
297
298     job_id = uuid.uuid4().hex
299     if not db_instance.create_job(job_id, req.video_path, req.segmenter,
300                                 req.player_pool, req.max_frames):
301         raise HTTPException(status_code=500, detail="Failed to persist job record.")
302     _get_executor().submit(_drain_due_jobs)
303     return JSONResponse(
304         status_code=202,
305         content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
306     )
307
308
309 @app.get("/api/jobs/{job_id}")
310 def get_job(job_id: str):
311     """Job state: {status, progress, result, reports}. `status` = execution state
312     (queued|running|done|failed); the pipeline verdict is result["status"]."""
313     job = db_instance.get_job(job_id)
314     if not job:
315         raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
316     result = job.get("result")
317     return {
318         "job_id": job["id"],
319         "status": job["status"],
320         "progress": job.get("progress"),
321         "video_id": job.get("video_id"),
322         "error": job.get("error"),
323         "result": result,
324         "reports": (result or {}).get("reports"),
325         "created_at": job.get("created_at"),
326         "started_at": job.get("started_at"),
327         "finished_at": job.get("finished_at"),
328     }
329
330 # --- Chunked upload (B2, PR-2) -----
331 # The chunked-upload island (in-process registry + helpers + the 4 endpoints) now lives
in
332 # backend.api.uploads as a self-contained APIRouter. It resolves the live startup config
via
333 # backend.main.global_config, so behavior ? route paths, status codes, body-size limits,
334 # sequential-part logic, and abort cleanup ? is unchanged.
335 from backend.api import uploads as uploads_api
336
337 app.include_router(uploads_api.router)
338
339
340 # --- Highlight download (B3, PR-2) -----
341

```

```

342     @app.get("/api/highlights/{video_id}/{filename}")
343     def download_highlight(video_id: str, filename: str):
344         """Stream a compiled highlight reel ? `filename` is the bare name given to
/api/compile
345         (which only writes into output_dir; the browser previously got back an unreachable
346         server-local path)."""
347         try:
348             name = safe_output_filename(filename)
349         except ValueError as e:
350             raise HTTPException(status_code=400, detail=str(e)) from e
351         if not db_instance.get_video(video_id):
352             raise HTTPException(status_code=404, detail="Indexed video record not found.")
353         output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
or "output"
354         path = os.path.join(output_dir, name)
355         try:
356             path = resolve_under_root(path, output_dir)
357         except ValueError as e:
358             raise HTTPException(status_code=400, detail=str(e)) from e
359         if not os.path.isfile(path):
360             raise HTTPException(status_code=404,
361                                 detail=f"No compiled file named '{name}'. Compile first via
/api/compile.")
362         media = "video/quicktime" if name.lower().endswith(".mov") else "video/mp4"
363         return FileResponse(path, media_type=media, filename=name)
364
365
366     @app.post("/api/query")
367     def query_play_segments(req: QueryRequest):
368         filters = {
369             "server": req.server,
370             "receiver": req.receiver,
371             "duration_min": req.duration_min,
372             "event_type": req.event_type
373         }
374         segments = db_instance.query_play_segments(req.video_id, filters)
375         return {"play_segments": segments}
376
377     @app.post("/api/compile")
378     def compile_highlights(req: CompileRequest):
379         video = db_instance.get_video(req.video_id)
380         if not video:
381             raise HTTPException(status_code=404, detail="Indexed video record not found.")
382
383         # Retrieve matching play segments from db
384         all_segments = db_instance.query_play_segments(req.video_id, {})
385         matched_segments = [s for s in all_segments if s["id"] in req.segment_ids]
386
387         if not matched_segments:
388             raise HTTPException(status_code=400, detail="No matching play segments found to
stitch.")
389
390         # Reject path traversal / absolute output names (os.path.join would otherwise let an
391         # absolute or '..'-laden output_filename write anywhere on disk).
392         try:
393             out_name = safe_output_filename(req.output_filename)
394         except ValueError as e:
395             raise HTTPException(status_code=400, detail=str(e)) from e
396
397         # The configured shot-count floor (stitching.min_shot_count) applies on the API
398         # path too ? parity with scripts/run_process_and_stitch.py. Segments WITHOUT
shot_count
399         # metadata are exempt: the floor filters known counts only, so explicitly
400         # requested manual/legacy segments can't silently vanish under the default floor.
401         stitching = getattr(global_config, "stitching", None) or StitchingConfig()

```

```

402     kept = []
403     for s in matched_segments:
404         meta = s.get("metadata") or {}
405         sc = meta.get("shot_count") if isinstance(meta, dict) else None
406         try:
407             if sc is not None and int(sc) < stitching.min_shot_count:
408                 continue
409             except (TypeError, ValueError):
410                 pass # unparseable count -> treat as unknown, keep
411             kept.append(s)
412         if len(kept) < len(matched_segments):
413             logger.info(f"[compile] stitching.min_shot_count={stitching.min_shot_count}:
dropped "
414                         f"{len(matched_segments) - len(kept)}/{len(matched_segments)}
segments below the floor.")
415         if not kept:
416             raise HTTPException(status_code=400,
417                                 detail=f"All {len(matched_segments)} matching segments fall
below "
418                                     f"stitching.min_shot_count={stitching.min_shot_count}.")
419
420         # Extract start/end time pairs
421         time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
422
423         # Run compiler with the configured stitching paddings + optional branding watermark
424         output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
425         or "output"
426         compiler = VideoCompiler(output_dir, begin_padding=stitching.begin_padding,
427                                 end_padding=stitching.end_padding,
428                                 watermark=stitching.watermark)
429         try:
430             out_path = compiler.compile_highlights(video["filepath"], time_pairs, out_name)
431             # `filepath` is server-local (not reachable by the browser); also return the
432             ready-to-use
433             # download URL so the SPA needn't hand-build it (P6 / PER_RALLY_SELECTION.md
434             §4.3).
435             download_url = f"/api/highlights/{quote(req.video_id)}/{quote(out_name)}"
436             return {"status": "success", "filepath": out_path, "download_url": download_url}
437         except Exception as e:
438             raise HTTPException(status_code=500, detail=f"Highlight stitching failed:
{str(e)}") from e
439
440     # --- CLI Debug Utility & Orchestration ---
441     def run_cli_diagnose_clip(video_path: str, timestamp: float):
442         """CLI debugging utility to examine segment properties around a timestamp."""
443         print("=" * 60)
444         print(f"DEBUGGING VIDEO CLIP: {video_path} at {timestamp}s")
445         print("=" * 60)
446
447         cfg = load_config() # legacy wrapper still works, returns dict for now
448
449         # 1. Validation check diagnostics
450         print("[DIAGNOSTIC] Running validation framework...")
451         results = registry.run_all(video_path, cfg)
452         for r in results:
453             status_symbol = "[PASS]" if r.passed else "[FAIL]"
454             print(f" {status_symbol} {r.validator_name}: {r.message}")
455             if r.details:
456                 print(f"      Details: {r.details}")
457
458         # 2. Local segment analysis around the timestamp (sampling +/- 5 seconds)
459         print("-" * 60)
460         print("[DIAGNOSTIC] Analyzing local motion around the timestamp...")
461         import cv2

```

```

458     cap = cv2.VideoCapture(video_path)
459     if not cap.isOpened():
460         print("[FAIL] OpenCV could not open video.")
461         return
462
463     fps = cap.get(cv2.CAP_PROP_FPS)
464     total_frames = cap.get(cv2.CAP_PROP_FRAME_COUNT)
465
466     start_frame = max(0, int((timestamp - 3.0) * fps))
467     end_frame = min(int(total_frames), int((timestamp + 3.0) * fps))
468
469     # Measure frame-by-frame changes in sample region
470     cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
471     prev_gray = None
472     motions = []
473
474     for _ in range(start_frame, end_frame):
475         ret, frame = cap.read()
476         if not ret:
477             break
478         gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
479         gray = cv2.GaussianBlur(gray, (21, 21), 0)
480
481         if prev_gray is not None:
482             diff = cv2.absdiff(prev_gray, gray)
483             _, thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)
484             motion_ratio = cv2.countNonZero(thresh) / (thresh.shape[0] *
thresh.shape[1])
485             motions.append(motion_ratio)
486             prev_gray = gray
487
488     cap.release()
489
490     if motions:
491         avg_motion = sum(motions) / len(motions)
492         # Support both legacy dict (from wrapper) and future direct model
493         if hasattr(cfg, "indexing"):
494             threshold = getattr(cfg.indexing, "motion_threshold", 0.08)
495         else:
496             threshold = cfg.get("indexing", {}).get("motion_threshold", 0.08) # type:
ignore[attr-defined]
497         print(f" Sample Frame Count: {len(motions)}")
498         print(f" Average motion index around timestamp: {round(avg_motion, 4)}
(threshold: {threshold})")
499         if avg_motion > threshold:
500             print(" Result: [ACTIVE PLAY] High motion detected. Segment classifies as
active play.")
501         else:
502             print(" Result: [DEAD TIME] Idle movement. Segment classified as dead
time.")
503     else:
504         print(" [ERROR] No visual frames were extracted in the sample range.")
505
506     print("=" * 60)
507
508     def main():
509         parser = argparse.ArgumentParser(description="Sports Highlight Indexer
Orchestrator")
510         parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
511         parser.add_argument("--output-dir", default="output", help="Directory where
processed clips/highlights are saved")
512         parser.add_argument("--setup", action="store_true", help="Launch dependency checker
and setup utility")
513         parser.add_argument("--debug-clip", type=float, help="Timestamp (in seconds) of a
video clip to run detailed diagnostics on")

```

```

514     parser.add_argument("--server", action="store_true", help="Run the FastAPI local API
server")
515     parser.add_argument("--port", type=int, default=8000, help="Port to run the FastAPI
server on")
516     parser.add_argument("--max-frames", type=int, help="Limit processing to the first N
sub-sampled frames for debugging")
517
518     available_segmenters = list(segmenter_registry.list_available().keys())
519     parser.add_argument("--segmenter", choices=available_segmenters, help=f"Choose rally
detection segmenter. Available: {available_segmenters}")
520     parser.add_argument("--trim-start", type=float, help="Start time in seconds for the
debug trim segment feature")
521     parser.add_argument("--trim-end", type=float, help="End time in seconds for the
debug trim segment feature")
522     parser.add_argument("--trim-output", help="Output filename for the debug trimmed
segment (saves in output-dir unless absolute path)")
523
524     args = parser.parse_args()
525
526     if args.setup:
527         # Invoke the diagnostics/bootstrap script (renamed from the old root
528         # setup.py to scripts/doctor.py so it no longer shadows the PEP 517
529         # build system). Resolve its path from this file so `indexer --setup`
530         # works regardless of the caller's cwd.
531         import subprocess
532         doctor_path = os.path.join(
533             os.path.dirname(os.path.abspath(__file__)),
534             "scripts", "doctor.py",
535         )
536         print(f"[INFO] Invoking diagnostics: {doctor_path} ...")
537         subprocess.run([sys.executable, doctor_path])
538         return
539
540     if args.trim_start is not None and args.trim_end is not None:
541         if not args.video_path:
542             print("[ERROR] Please provide --video-path to extract a segment.")
543             return
544
545         # Determine output path
546         out_filename = args.trim_output or "trimmed_segment.mp4"
547         if os.path.isabs(out_filename):
548             out_path = out_filename
549             out_dir = os.path.dirname(out_path)
550             out_name = os.path.basename(out_path)
551         else:
552             out_dir = args.output_dir
553             out_path = os.path.join(out_dir, out_filename)
554             out_name = out_filename
555
556         os.makedirs(out_dir, exist_ok=True)
557         print(f"[CLI] Extracting debug segment from {args.trim_start}s to
{args.trim_end}s from {args.video_path}...")
558
559         # global_config isn't initialized this early; load the typed config so the trim
560         # clip carries the same configured stitching paddings + watermark as a real
561         # highlight (watermark default-on).
562         stitching = load_app_config().stitching
563         compiler = VideoCompiler(out_dir, begin_padding=stitching.begin_padding,
564                                 end_padding=stitching.end_padding,
565                                 watermark=stitching.watermark)
566         try:
567             res_path = compiler.compile_highlights(args.video_path, [(args.trim_start,
568                             args.trim_end)], out_name)
569             print(f"[SUCCESS] Trimmed segment saved to: {res_path}")
570         except Exception as e:

```

```

569         print(f"[ERROR] Failed to compile trimmed segment: {e}")
570     return
571
572     if args.debug_clip is not None:
573         if not args.video_path:
574             print("[ERROR] Please provide --video-path to debug a specific clip.")
575             return
576         run_cli_diagnose_clip(args.video_path, args.debug_clip)
577     return
578
579     # Initialize Global Config and DB
580     global global_config, db_instance
581     global_config = load_app_config() # returns typed AppConfig
582
583     # M5: per-profile startup checks ? fail fast on an incoherent ACTIVE profile (e.g. a
584     # cloud-serving profile pointed at local-only storage) BEFORE any work. A strict no-
585     # op when
586     # no deployment.profile is selected (the default), so today's behaviour is
587     # unchanged. No GPU
588     # probe here (the server stays torch-light; the detector healthcheck is the GPU
589     # authority).
590     try:
591         enforce_startup_checks(global_config, logger=logger)
592     except StartupCheckError as e:
593         logger.error("[STARTUP] refusing to start: %s", e)
594         sys.exit(1)
595
596     # The CLI --output-dir overrides the configured output directory. It now lives
597     # on the typed model (OutputConfig.output_dir), so every consumer ? including
598     # /api/compile ? reads the same value instead of a write-only runtime hack.
599     global_config.output_dir = args.output_dir
600     os.makedirs(global_config.output_dir, exist_ok=True)
601
602     db_path = os.path.join(global_config.output_dir,
603                             global_config.output.db_name)
604     db_instance = Database(db_path)
605
606     # Crash recovery (#16): jobs left queued/running by a previous process are dead ?
607     # the executor is in-process. Mark them failed so pollers see a terminal state.
608     swept = db_instance.fail_stale_jobs()
609     if swept:
610         logger.warning(f"[JOBS] Swept {swept} stale job(s) from a previous run ?
611         failed.")
612
613     # CLI processing mode (no web UI needed)
614     if args.video_path and not args.server:
615         print(f"[CLI] Processing video file: {args.video_path}")
616         # M2: bare/local path = identity (today's behaviour); a bucket/Drive URI is
617         # fetched to scratch.
618         # The local-existence fast-fail only applies to a local input (stat the RESOLVED
619         # key, not the
620         # raw URI); a remote fetch fails loudly. An unsupported scheme is a clean error,
621         # not a crash.
622         storage_cfg = getattr(global_config, "storage", None)
623         try:
624             input_ref = storage.resolve_input_ref(args.video_path, storage_cfg)
625         except ValueError as e:
626             print(f"[FAIL] {e}")
627             return
628         if input_ref.backend == "local" and not os.path.exists(input_ref.key):
629             print(f"[FAIL] Video file not found: {args.video_path}")
630             return
631         local_video = storage.acquire_local_input(args.video_path, storage_cfg)
632
633         video_id = compute_video_id(local_video)

```



```

626         was_reingest = db_instance.get_video(video_id) is not None
627
628         # Validate (with-context variant surfaces ffprobe_meta for the report)
629         print("[CLI] Validating...")
630         val_results, val_context = registry.run_all_with_context(local_video,
631 global_config)
632
633         failures = [r for r in val_results if not r.passed]
634
635         # Save video status
636         status = "failed" if failures else "processed"
637         db_instance.add_video(video_id, args.video_path, status, [r.model_dump() for r
638 in val_results])
639
640         seg_name = args.segmenter or getattr(getattr(global_config, "indexing", None),
641 "default_segmenter", "motion")
642         segmenter_actual = None
643         segmentation_ran = False
644         try:
645             print("\n" + "="*40)
646             print("VALIDATION SUMMARY")
647             print("="*40)
648             for r in val_results:
649                 status_symbol = "[PASS]" if r.passed else "[FAIL]"
650                 print(f"{status_symbol} {r.validator_name}: {r.message}")
651             print("="*40 + "\n")
652
653             if failures:
654                 print("[FAIL] Video failed checks. Indexing halted.")
655                 return
656
657             # Index
658             segmenter_cls = segmenter_registry.get(seg_name)
659             if not segmenter_cls:
660                 print(f"[FAIL] Segmenter '{seg_name}' not found.")
661                 return
662
663             print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
664             segmenter_actual = seg_name
665             play_wm, cand_wm = db_instance.segment_watermarks(video_id)
666             segmenter = segmenter_cls(db_instance, global_config)
667             result = segmenter.process_video(video_id, local_video,
668 max_frames=args.max_frames)
669             segmentation_ran = True
670
671             if isinstance(result, Failure):
672                 print(f"[FAIL] Segmenter '{seg_name}' failed: {result.message}")
673                 print("[FAIL] Indexing halted for video.")
674                 segments = []
675             else:
676                 segments = result.value if hasattr(result, "value") else result
677                 print(f"[SUCCESS] Process complete. Indexed {len(segments)} play
678 segments inside database: {db_path}")
679                 # Destroy-AFTER-confirm: keep new on success, restore prior on
680 empty/failure.
681                 _finalize_reingest(video_id, segments, play_wm, cand_wm)
682             finally:
683                 # Always emit the per-video observability report. Never raises.
684                 paths = emit_indexer_report(
685                     db=db_instance, video_id=video_id, video_path=args.video_path,
686 config=global_config,
687                     val_results=val_results, val_context=val_context,
688                     segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
689                     was_reingest=was_reingest, segmentation_ran=segmentation_ran,
690 )
691                 if paths:

```

```

684         print(f"[REPORT] Technical report : {paths.get('technical_md')}")
685         print(f"[REPORT] Customer summary : {paths.get('customer_md')}")
686         print(f"[REPORT] Machine JSON      : {paths.get('json')}")
687     return
688
689     # Start Local Server Mode
690     if args.server or not args.video_path:
691         print(f"[SERVER] Starting local FastAPI server on
http://localhost:{args.port}...")
692         # Resume self-scheduled work orphaned by the restart (EXECUTION_POLICY.md P3):
the
693         # in-process wake-timer for any parked 'waiting' job died with the old process,
so
694         # kick one drain ? it reclaims due jobs and re-arms the wake-timer for the rest.
695         _get_executor().submit(_drain_due_jobs)
696         uvicorn.run(app, host="127.0.0.1", port=args.port)
697
698     if __name__ == "__main__":
699         # `python -m backend.main` runs THIS file as `__main__`, but `job_worker`/`uploads`
resolve the
700         # shared mutable state (`db_instance`, `global_config`) via `import backend.main` ?
a SEPARATE
701         # module object. If `main()` runs here it configures `__main__`'s globals, leaving
702         # `backend.main.db_instance` = None; the async-job drain then crashes silently on
703         # `None.claim_due_job()` and every submitted job hangs in 'queued' forever. Dispatch
through
704         # `backend.main` so `main()` configures the module everyone actually reads. (No
recursion:
705         # imported as `backend.main`, `__name__` != "__main__", so this block doesn't re-
fire.)
706         import backend.main as _bm
707
708         _bm.main()
709

```

13. assistant (2026-06-21T10:18:48.116Z)

Now let me dig into the critical question: does the stub trajectory actually produce non-empty windows deterministically? Let me look at the stub runner and the segmenter's process_video path to understand whether `len(local) >= 1` is meaningful and whether `video\_id` affects the trajectory.

14. user (2026-06-21T10:18:48.986Z)

```

1     """CPU stub shuttle-trajectory runner ? "mock a GPU with a CPU".
2
3     A ``DetectorRunner`` that produces a deterministic ``Frame,Visibility,X,Y`` trajectory
4     CSV with no GPU, no WSL, no model weights ? so the whole trajectory-hybrid pipeline
5     (``wasb_hybrid`` / ``tracknet_hybrid``) can run end-to-end on a CPU-only box (CI, a CPU
6     container, a laptop) for regression testing. The person/yolo path already CPU-falls-back
7     (``use_gpu=False``); this closes the last GPU/WSL-only gap so the entire pipeline is
8     runnable ? and regression-gateable ? without a GPU.
9
10    Two modes:
11    * replay ? emit an existing ``Frame,Visibility,X,Y`` CSV verbatim (drive the
12    pipeline with a committed/golden trajectory; deterministic + cross-OS).
13    * synth (default) ? generate a deterministic synthetic trajectory of
14    ``num_rallies`` in-flight bursts (visible + fast-moving frames) separated by dead
15    gaps, so windowing yields a stable, known number of candidate windows.
16
17    This is the CPU seam the ``DetectorRunner`` ABC was built for (see ``base.py`` ?
18    "Optional Linux-native path") and the seed of the future ``LinuxNativeRunner``
19    (M3.5, ``docs/DECOUPLED_COMPUTE/04-BUILD-AND-TEST-PLAN.md``). It is selected via
20    ``indexing.detector_impl="stub"`` or the ``RALLY_DETECTOR_IMPL=stub`` env var; the real
21    WSL runners stay the default, so production behaviour is unchanged.

```

```

22     """
23     import os
24     import csv as _csv
25     import shutil
26     import logging
27     from dataclasses import dataclass
28     from typing import Any, Dict, List, Optional, Tuple
29
30     from backend.pipeline.detectors.base import DetectorRunner
31     # Reuse the model-agnostic CSV parsing + windowing (no torch/tf pulled here).
32     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
33
34     logger = logging.getLogger(__name__)
35
36
37     @dataclass
38     class StubConfig:
39         """Settings for the CPU stub runner (built from ``config.json`` ->
40         ``indexing.stub``)."""
41         fps: float = 30.0
42         frame_width: float = 1280.0
43         num_rallies: int = 2
44         rally_seconds: float = 4.0
45         gap_seconds: float = 3.0          # dead gap between rallies (> merge_gap so
46         windows don't merge)
47         lead_seconds: float = 2.0        # dead lead-in
48         step_px: float = 12.0            # per-frame shuttle displacement during a rally
49         (>> velocity_thresh)
50         replay_csv: Optional[str] = None # if set + exists, emit this CSV verbatim instead
51         of synthesising
52
53     @classmethod
54     def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "StubConfig":
55         s = (idx_cfg or {}).get("stub", {}) or {}
56         return cls(
57             fps=float(s.get("fps", 30.0)),
58             frame_width=float(s.get("frame_width", 1280.0)),
59             num_rallies=int(s.get("num_rallies", 2)),
60             rally_seconds=float(s.get("rally_seconds", 4.0)),
61             gap_seconds=float(s.get("gap_seconds", 3.0)),
62             lead_seconds=float(s.get("lead_seconds", 2.0)),
63             step_px=float(s.get("step_px", 12.0)),
64             replay_csv=s.get("replay_csv"),
65         )
66
67     class StubDetectorRunner(DetectorRunner):
68         """Deterministic CPU trajectory runner (no GPU/WSL). See the module docstring."""
69
70         def __init__(self, cfg: Optional[StubConfig] = None):
71             self.cfg = cfg or StubConfig()
72
73         def healthcheck(self) -> Tuple[bool, str]:
74             return True, "stub CPU detector (no GPU/WSL)"
75
76         def _rows(self) -> List[Tuple[int, int, float, float]]:
77             """Deterministic ``(frame, visible, x, y)`` rows: ``num_rallies`` in-flight
78             bursts
79             separated by dead gaps. During a rally the shuttle oscillates ``step_px``/frame
80             ?
81             well above the default 0.02 velocity threshold ? so every rally frame is
82             "active"
83             and each burst becomes exactly one candidate window."""
84             cfg = self.cfg
85             rows: List[Tuple[int, int, float, float]] = []

```

```

80         frame = 0
81         y = cfg.frame_width / 4.0
82         base_x = cfg.frame_width / 2.0
83         for _ in range(int(cfg.lead_seconds * cfg.fps)):
84             rows.append((frame, 0, 0.0, 0.0))
85             frame += 1
86         for r in range(cfg.num_rallies):
87             for i in range(int(cfg.rally_seconds * cfg.fps)):
88                 x = base_x + (cfg.step_px if i % 2 else -cfg.step_px)
89                 rows.append((frame, 1, x, y))
90                 frame += 1
91             if r != cfg.num_rallies - 1:
92                 for _ in range(int(cfg.gap_seconds * cfg.fps)):
93                     rows.append((frame, 0, 0.0, 0.0))
94                     frame += 1
95         return rows
96
97     def run_predict(self, video_win_path: str, output_win_dir: str,
98                     video_id: Optional[str] = None) -> Optional[str]:
99         """Write a deterministic trajectory CSV and return its path (never touches the
100 GPU/WSL)."""
101         os.makedirs(output_win_dir, exist_ok=True)
102         stem = os.path.splitext(os.path.basename(str(video_win_path).replace("\\",
103 "/"")))[0]
104         key = f"{stem}__{video_id[:12]}" if video_id else (stem or "stub")
105         out_csv = os.path.join(output_win_dir, f"{key}_stub.csv")
106
107         if self.cfg.replay_csv and os.path.exists(self.cfg.replay_csv):
108             shutil.copyfile(self.cfg.replay_csv, out_csv)
109             logger.info("Stub runner: replayed trajectory %s -> %s",
110 self.cfg.replay_csv, out_csv)
111             return out_csv
112
113         rows = self._rows()
114         with open(out_csv, "w", newline="") as f:
115             w = _csv.writer(f)
116             w.writerow(["Frame", "Visibility", "X", "Y"])
117             for frame, vis, x, y in rows:
118                 w.writerow([frame, vis, f"{x:.3f}", f"{y:.3f}"])
119             logger.info("Stub runner: wrote %d synthetic trajectory rows -> %s", len(rows),
120 out_csv)
121         return out_csv
122
123
124     # Model-agnostic CSV parsing + windowing ? identical to the real runners.
125     parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
126     trajectory_to_action_windows =
127 staticmethod(TrackNetRunner.trajectory_to_action_windows)
128
129
130     def synth_trajectory_from_labels(labels_csv: str, out_csv: str, *, fps: float = 30.0,
131                                     frame_width: float = 1280.0, tail_pad: float = 2.0,
132                                     step_px: float = 12.0) -> str:
133         """CPU "mock GPU" for TRAINING: write a deterministic Frame,Visibility,X,Y
134 trajectory whose
135 in-flight (visible + oscillating) frames cover each labeled rally window and whose
136 dead frames
137 cover the gaps ? so the synthetic trajectory *encodes* the rally labels.
138
139 This lets the Gen-0 harness train + leave-one-video-out eval end-to-end on coherent
140 data with
141 no GPU/WSL, validating the whole training pipeline before the real WASB detector
142 lands. (The
143 learned model is meaningful for plumbing, not for production ? the trajectory is
144 synthetic.)
145 """

```

```

135     rallies: List[Tuple[float, float]] = []
136     with open(labels_csv, newline="", encoding="utf-8") as f:
137         for row in _csv.DictReader(f):
138             try:
139                 rallies.append((float(row["start_time"]), float(row["end_time"])))
140             except (KeyError, ValueError):
141                 continue
142     duration = (max(e for _, e in rallies) + tail_pad) if rallies else 10.0
143     nframes = max(1, int(duration * fps))
144     base_x = frame_width / 2.0
145     y = frame_width / 4.0
146     os.makedirs(os.path.dirname(out_csv) or ".", exist_ok=True)
147     with open(out_csv, "w", newline="", encoding="utf-8") as f:
148         w = _csv.writer(f)
149         w.writerow(["Frame", "Visibility", "X", "Y"])
150         for i in range(nframes):
151             t = i / fps
152             if any(s <= t < e for s, e in rallies):
153                 x = base_x + (step_px if i % 2 else -step_px)
154                 w.writerow([i, 1, f"{x:.3f}", f"{y:.3f}"])
155             else:
156                 w.writerow([i, 0, "0.000", "0.000"])
157     return out_csv
158

```

15. assistant (2026-06-21T10:18:53.390Z)

Now I understand the stub. The stub's trajectory is generated purely from `StubConfig` defaults (num_rallies=2), independent of the video bytes or `video\_id`? `video\_id` only affects the output CSV *filename* (`key`), not the rows. So the trajectory is genuinely identical across arms. Let me verify the windowing function and how the segmenter wires the stub, plus check the `\_\_init\_\_.py` exports and whether `device='cuda'` default vs the config's wasb device matters in the parity test path.

16. user (2026-06-21T10:18:54.789Z)

```

1     """Configuration package.
2
3     Public API:
4         from backend.config import load_config, AppConfig, Config
5
6     This package is the canonical place for all configuration access.
7     """
8
9     from .loader import load_config, save_default_config
10    from .models import ( # re-export common ones
11        AppConfig,
12        Config,
13        DeploymentConfig,
14        IndexingConfig,
15        StitchingConfig,
16        StorageConfig,
17        ValidationConfig,
18    )
19    from .presets import PROFILE_PRESETS, probe_cuda_available, suggest_profile
20    from .startup import (
21        StartupCheck,
22        StartupCheckError,
23        enforce_startup_checks,
24        run_startup_checks,
25    )
26
27    __all__ = [
28        "load_config",
29        "save_default_config",

```

```

30     "AppConfig",
31     "Config",
32     "DeploymentConfig",
33     "IndexingConfig",
34     "StitchingConfig",
35     "StorageConfig",
36     "ValidationConfig",
37     "PROFILE_PRESETS",
38     "suggest_profile",
39     "probe_cuda_available",
40     "StartupCheck",
41     "StartupCheckError",
42     "enforce_startup_checks",
43     "run_startup_checks",
44 ]
45

```

17. user (2026-06-21T10:18:55.675Z)

```

281: def trajectory_to_action_windows(
282-     points: List[TrajectoryPoint],
283-     fps: float,
284-     frame_width: float,
285-     chunk_start: float = 0.0,
286-     velocity_thresh: float = 0.02,
287-     merge_gap: float = 2.0,
288-     min_window_duration: float = 2.0,
289-     chunk_index: Optional[int] = None,
290-     max_window_duration: float = 0.0,
291-     min_split_gap: float = 0.5,
292-     window_hard_cap: bool = False,
293-     window_inactivity_end: float = 0.0,
294-     inactivity_min_density: float = 0.34,
295-     inactivity_rally_thresh: float = 0.08,
296-     window_blob_fallback: bool = False,
297-     window_blob_factor: float = 4.0,
298- ) -> List[Dict[str, Any]]:
299-     """Convert a shuttle trajectory into candidate rally windows.
300-
301-     A frame is "active" when the shuttle is visible AND its inter-frame
302-     displacement (normalised by frame width, per second) exceeds
303-     ``velocity_thresh`` ? i.e. the shuttle is in flight. Active frames within
304-     ``merge_gap`` seconds of each other are grouped into one window; short
305-     windows are dropped. Mirrors HybridYoloSegmenter's windowing so the dict
306-     shape feeding the AI-handoff phase is identical.
307-
308-     ``max_window_duration`` (0 = OFF, the default ? behaviour unchanged): a guard
309-     against
310-     the *over-merge* failure where "shuttle moving" is conflated with "rally in
311-     progress"
312-     and a single window swallows several rallies + the dead time between them. When set,
313-     any
314-     window longer than this is recursively SPLIT at its largest internal inactivity gap
315-     (the
316-     longest stretch with no in-flight shuttle ? the most likely rally boundary),
317-     provided
318-     that gap is at least ``min_split_gap`` seconds. This is the bounded-windowing fix
319-     from
320-     ``docs/RALLY_QUALITY_RESEARCH.md`` §6 (W1); it never merges, only cuts, so recall
321-     cannot
322-     drop, and it is config-gated default-OFF until measured on the golden set.
323-
324-     ``window_hard_cap`` (default OFF) makes ``max_window_duration`` a TRUE cap: when an
325-     over-long window has NO internal inactivity gap to split on (a continuously-active
326-     trajectory ? the shuttle never stops moving, e.g. the real-footage ``mahadevpura-1``

```

320- blob where the gap-splitter alone left a single 174 s window), it is recursively
 321- hard-chopped at its temporal midpoint until every piece is ? the cap. Still only
 cuts
 322- (never merges), so recall cannot drop; gated default-OFF until measured.
 323-
 324- ``window\_blob\_fallback`` (default OFF ? the R5 last-resort blob splitter): unlike
 325- ``window\_hard\_cap`` (which midpoint-chops BEFORE the R4 trim, so it fires on every
 gap-less
 326- over-cap window and over-segments normal clips), this runs AFTER the R4 inactivity
 trim and

18. assistant (2026-06-21T10:19:01.531Z)

The signature confirms `trajectory\_to\_action\_windows` has no profile/compute/storage parameter ? so the L4a structural guard is meaningful. The banned set `{"profile", "compute\_target", "deployment", "storage", "backend"}` correctly excludes the real params (fps, velocity_thresh, etc.).

Now let me verify two things the parity test relies on: (1) that the stub is actually wired into `WasbHybridSegmenter.process\_video` with `detector\_impl="stub"` so the trajectory is the stub's deterministic 2-rally output, and (2) that the device default doesn't break things. Let me check how the segmenter resolves the stub runner and that the stub's default config (num_rallies=2, etc.) yields ?1 window. Let me also confirm whether the worker test config (no profile) vs the in-process ref (profile=truly-local) could diverge ? the worker test config has NO profile but the ref uses `truly-local`.

19. user (2026-06-21T10:19:03.607Z)

```
62:         f"detector_impl='native' is not supported for the "
64:         f"Linux-native runner. Use detector_impl='auto' (WSL) or 'stub'."
66:     def _resolve_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
67:         """Resolve the detector runner, honouring the CPU-stub and Linux-native overrides.
69:         ``RALLY_DETECTOR_IMPL`` env var (takes precedence) or ``indexing.detector_impl``:
70:         ``stub`` swaps in a deterministic CPU runner (no GPU/WSL) so the whole pipeline
77:         impl = (os.environ.get("RALLY_DETECTOR_IMPL") or idx_cfg.get("detector_impl") or
"auto").lower()
78:         if impl == "stub":
79:             from backend.pipeline.detectors.stub_runner import StubDetectorRunner, StubConfig
80:             logger.info("%s: detector_impl=stub ? CPU mock runner (no GPU/WSL).",
82:             return StubDetectorRunner(StubConfig.from_indexing_cfg(idx_cfg)),
{"detector_impl": "stub"}
84:             logger.info("%s: detector_impl=native ? Linux-native runner (no WSL).",
191:         # _resolve_runner honours the CPU-stub override (RALLY_DETECTOR_IMPL /
192:         # indexing.detector_impl="stub") so the pipeline runs GPU/WSL-free; otherwise
195:         runner, runner_params = self._resolve_runner(idx_cfg)
197:         # e.g. detector_impl="native" for a family without a native runner (tracknet, or
```

20. assistant (2026-06-21T10:19:10.194Z)

The stub is resolved via `detector\_impl="stub"` from `idx\_cfg`, and `StubConfig.from\_indexing\_cfg` reads from `idx\_cfg.get("stub", {})` ? which is empty in the test (no `indexing.stub` key), so it uses the defaults (num_rallies=2). This confirms the trajectory is identical and deterministic across all arms.

Now, the key subtlety the prompt flags: in `test\_worker\_intervals\_match\_inprocess`, the worker config has **no profile**, but the in-process `ref` uses `profile="truly-local"`. The L4b test already proves no-profile == truly-local produce identical ranges, so this is consistent. But let me verify the worker run actually invokes the stub windowing path (not some divergent path) and confirm there's no risk the comparison is `[] == []`. Let me check what `velocity\_thresh` etc. defaults the segmenter passes, and run the actual tests to ground my analysis.

21. user (2026-06-21T10:19:20.948Z)

[100%]

.....

25 passed in 7.50s

22. assistant (2026-06-21T10:19:24.431Z)

All 25 pass. Now let me prove the non-vacuity concern empirically: how many windows does the stub actually produce, and would the ``len(local) >= 1`` assertion catch a regression? Let me instrument the actual in-process path to see the real segment count, and confirm the worker's `intervals.csv` is non-empty (so ``worker == ref`` isn't ``[] == []``).

23. user (2026-06-21T10:19:36.756Z)

SEGMENT COUNT: 2

RANGES: [(2.033, 5.967), (9.033, 12.967)]

[mov,mp4,m4a,3gp,3g2,mj2 @ 0000014e54a3e580] moov atom not found

24. assistant (2026-06-21T10:19:42.562Z)

The stub produces exactly 2 non-empty segments with specific ranges ``[(2.033, 5.967), (9.033, 12.967)]``. So:

- ``len(local) >= 1`` is satisfied non-vacuously (actual count is 2), and the comparison ``local == mock == none`` compares two specific 2-tuple lists, not empty lists.
- The ``worker == ref`` comparison is genuinely comparing ``[(2.033, 5.967), (9.033, 12.967)]`` against itself, not ``[] == []``.

Now let me probe the most important false-confidence question: ****would the L4b assertion actually FAIL if a profile genuinely leaked into the core?**** Let me test whether ``len(local) >= 1`` would catch the failure mode where the stub produces zero segments (e.g., if `process_video` returned a `Failure`). Let me also check: does ``process_video`` return a ``Failure`` object on the fake video, and does ``_ranges`` handle that? Let me verify the result type handling.

25. user (2026-06-21T10:19:43.457Z)

```
150         logger.error(str(e))
151         return []
152
153         idx_cfg = self.config.get("indexing", {})
154         # Fully-local, NO-AI mode (default OFF): the trajectory candidate windows become
the
155         # rallies directly ? Phase 3's AI handoff is skipped, so no provider / API key
is
156         # needed and nothing is uploaded. Used to run the local detector standalone
(e.g. to
157         # measure it against the Gemini path, or to avoid the cloud entirely). With
158         # FusionSegmenter the windows are still Gate-A/B-filtered via
`_select_for_handoff`.
159         skip_ai = bool(idx_cfg.get("skip_ai_handoff", False))
160         provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
161         if skip_ai:
162             model_name = "local-noai"
163             logger.info("%s in FULLY-LOCAL mode (skip_ai_handoff=True): candidate
windows will "
164                         "be emitted as rallies; no %s handoff, no API key needed.",
165                         label, provider_name)
166         else:
167             model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
168             api_key_env_var = f"{provider_name.upper()}_API_KEY"
169             api_key = os.environ.get(api_key_env_var)
170             if not api_key:
171                 from backend.pipeline.segmenters._keyhint import api_key_hint
172                 logger.error(
173                     f"{api_key_env_var} environment variable is not set! "
```



```

174         f"To run the {provider_name} handoff, set your API key. "
175         + api_key_hint(api_key_env_var)
176     )
177     return []
178     try:
179         provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
180     except KeyError as e:
181         logger.error(str(e))
182         return []
183
184     video_duration = get_video_duration(video_path)
185     if video_duration <= 0:
186         logger.error("Invalid video duration.")
187         return []
188     fps, frame_width = self._probe_fps_width(video_path)
189
190     # --- Runner config + windowing thresholds ---
191     # _resolve_runner honours the CPU-stub override (RALLY_DETECTOR_IMPL /
192     # indexing.detector_impl="stub") so the pipeline runs GPU/WSL-free; otherwise
193     # it delegates to the subclass _build_runner (the real WSL/native runner).
194     try:
195         runner, runner_params = self._resolve_runner(idx_cfg)
196     except NotImplementedError as e:
197         # e.g. detector_impl="native" for a family without a native runner
(tracknet, or
198         # fusion with a tracknet substrate). Refuse cleanly (no rallies) instead of
crashing.
199         logger.error("%s: %s", label, e)
200         return []
201
202     velocity_thresh = idx_cfg.get(self.family, {}).get("velocity_threshold", 0.02)
203     merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
204     min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)
205     # Bounded-windowing over-merge fix (W1): 0 = OFF (unchanged). See
IndexingConfig.
206     max_window_duration = idx_cfg.get("max_window_duration", 0.0)
207     window_min_split_gap = idx_cfg.get("window_min_split_gap", 0.5)
208     window_hard_cap = idx_cfg.get("window_hard_cap", False)
209     window_inactivity_end = idx_cfg.get("window_inactivity_end", 0.0)

```

26. assistant (2026-06-21T10:19:49.061Z)

This is a critical observation. The early-return paths in `process\_video` all `return []` (empty list) on failure ? e.g. missing API key, invalid duration, unresolvable runner. This is exactly the false-confidence risk: ****if the stub silently broke and returned `[]`, the L4b equality `local == mock == none` would still pass (all three `[]`), and only `len(local) >= 1` is the guard against vacuity.****

The `len(local) >= 1` assertion IS present and IS the safety net. Let me verify it would actually fire by simulating the empty-result case, to confirm the guard is real and not bypassed.

27. user (2026-06-21T10:19:50.840Z)

GUARD WORKS: `len(local) >= 1` catches the all-empty case

28. assistant (2026-06-21T10:20:00.480Z)

The L4b guard is sound. However, I notice the L4c worker test (``test_worker_intervals_match_inprocess``) has a subtler gap. Let me re-examine it: ``assert worker == ref``. If both `worker` and `ref` were empty, this would pass vacuously. The `ref` comes from ``_inprocess_ranges(...truly-local...)`` which has NO ``len >= 1`` guard of its own (the guard only lives in the L4b test function, not in the ``_inprocess_ranges`` helper). Let me

confirm whether L4c has any non-emptiness assertion.

Looking back at the test (lines 99-139): there is **no** ``assert ref`` or ``assert worker`` non-emptiness check in L4c. If the stub trajectory ever regressed to producing zero windows, L4b would catch it (via ``len(local) >= 1``), but L4c's ``worker == ref`` would pass vacuously as ``[] == []``. This is a real (if minor) false-confidence gap ? L4c relies on L4b running to catch emptiness, but as an independent assertion it could pass on empty.

Let me also examine whether the worker test's ``cmd_infer`` could ``return 0`` while writing an empty `intervals.csv`, and whether ``rc == 0`` provides any protection. Let me check the worker's exit codes and confirm one more concern: the worker config writes no profile, while the cloud-serving fatal test. Let me verify the ``model_validate`` default for ``deployment.profile`` and that ``_cfg``'s ``device="cuda"`` default interacts correctly with ``cuda_available=None`` skip.

29. user (2026-06-21T10:20:03.656Z)

```
default profile: ''
default storage backend: 'local'
default detector_impl: 'auto'
wasb device default: 'cuda'
```

30. assistant (2026-06-21T10:20:12.056Z)

Defaults confirmed. Now let me scrutinize a few specific test assertions for weakness in ``test_startup_checks.py``:

1. ``test_truly_local_cloud_storage_warns_unusual`` (line 78-81): asserts ``"STORAGE_UNUSUAL" in _codes(checks)`` ? uses ``in`` rather than ``==``. With ``device="auto"`` and ``cuda_available=True``, no GPU warning fires, so checks should be exactly ``{STORAGE_UNUSUAL}``. The ``in`` is weaker than ``==`` but not wrong. Let me verify other ``in``-based assertions don't mask a missing/extra check.

2. ``test_cloud_serving_cpu_device_warns`` (117-120): ``device="cpu"``, ``cuda_available=True``. The ``CLOUD_CPU_DEVICE`` branch fires. But note the ``elif cuda_available is False`` ? with ``cuda_available=True`` that `elif` is skipped. Uses ``in``. Fine.

Let me check the one assertion that could be misleading: ``test_cloud_serving_no_gpu_warns`` uses ``device="cuda"``, ``cuda_available=False``. Both the storage check (gcs, ok) and ``CLOUD_CUDA_NOT_VISIBLE`` should fire. Let me confirm there's no ``CREDS_UNCONFIRMED`` leaking (env has `K_SERVICE` so creds OK). Let me verify the exact code sets for the ``in``-based tests to see if any extra/unexpected code is being silently tolerated.

31. user (2026-06-21T10:20:18.981Z)

```
truly-local gcs auto cuda=True: ['STORAGE_UNUSUAL']
cloud-serving gcs cpu cuda=True K_SERVICE: ['CLOUD_CPU_DEVICE']
cloud-serving gcs cuda cuda=False K_SERVICE: ['CLOUD_CUDA_NOT_VISIBLE']
cpu-mock native: ['MOCK_NONSTUB_DETECTOR']
cloud-serving local cuda=True creds: ['STORAGE_INCOHERENT']
```

32. assistant (2026-06-21T10:20:27.438Z)

All the ``in``-based assertions happen to match singleton sets, so they're not currently masking anything ? though ``==`` would be stricter. These are nits at most.

Now let me verify the DEFAULT-OFF byte-identical claim more rigorously. The prompt says "when `deployment.profile==''` NOTHING changes vs pre-M5 (byte-identical)". The test ``test_no_profile_is_strict_noop`` checks ``run_startup_checks(...) == []``. But the real default-off contract is about the **production entry points** (main.py / worker) being no-ops. Let me check: in ``backend/main.py`` server path, ``enforce_startup_checks(global_config, logger=logger)`` is called with NO ``cuda_available`` (defaults to None) and NO env. With `profile=""`, ``run_startup_checks`` returns ``[]`` immediately, so no logging, no raise. Good.

Let me verify one potential false-confidence issue in the L4c test: does ``cmd_infer`` with no profile actually run the startup-check hook as a no-op (not interfering)? And critically ? is

there a test proving the `**server main() path**` no-ops on default? The startup-checks tests cover ``run_startup_checks`/`enforce_startup_checks`` directly and the worker ``cmd_infer` rc2` path, but I don't see a test that the server's ``main()` sys.exit(1)` path fires (or that it's a no-op when `profile==""`). Let me check if that's covered anywhere.

33. user (2026-06-21T10:20:30.133Z)

```
tests/test_startup_checks.py
---grep main sys.exit startup in tests---
```

34. assistant (2026-06-21T10:20:39.238Z)

So the server ``main()`` enforcement (`sys.exit(1)` on fatal, and the no-op on default) has `**no test coverage**` ? only the worker's ``cmd_infer` rc2` path is tested (``test_worker_cmd_infer_aborts_rc2_on_incoherent_profile``). The server is the symmetric entry point with its own ``try/except StartupCheckError ? sys.exit(1)`` wiring at `main.py:587-591`. That wiring is genuinely untested. This is within my lens (test rigor / coverage of the load-bearing fatal path).

Let me also double-check the L4c worker test for one more subtle false-confidence point: it mocks ``validator_registry.run_all_with_context`` to return a passing validator, but does it actually reach segmentation? If validators failed, ``cmd_infer`` returns rc 2 and writes no `intervals.csv`, and ``next(out.rglob("intervals.csv"))`` would raise ``StopIteration`` ? which would be a loud failure, not a silent pass. Good, that's protected by ``assert rc == 0`` first. Let me confirm the ``rc == 0`` and the `rglob` both genuinely guard.

Let me verify the worker actually produces the same 2 ranges to be 100% sure L4c isn't vacuously comparing empties, by running the worker path directly.

35. user (2026-06-21T10:20:51.880Z)

```
[worker] pulled C:\Users\avidu\AppData\Local\Temp\tmpsonq01gb\w.mp4 ->
C:\Users\avidu\AppData\Local\Temp\tmpsonq01gb\scratch\in.mp4
[worker] infer done: 2 segment(s); pushed 2 artifact(s):
  C:\Users\avidu\AppData\Local\Temp\tmpsonq01gb\bucket/outputs/752e5362eeb38fc8c6bd7a7fca793cb
0/intervals.csv
  C:\Users\avidu\AppData\Local\Temp\tmpsonq01gb\bucket/outputs/752e5362eeb38fc8c6bd7a7fca793cb
0/report.json
rc: 0
csv: ['C:\\Users\\avidu\\AppData\\Local\\Temp\\tmpsonq01gb\\bucket\\outputs\\752e5362eeb38fc8c6b
d7a7fca793cb0\\intervals.csv']
worker ranges: [(2.033, 5.967), (9.033, 12.967)]
```

36. assistant (2026-06-21T10:20:58.864Z)

Confirmed: the worker produces 2 segments with identical ranges ``[(2.033, 5.967), (9.033, 12.967)]`` to the in-process run. So L4c's ``worker == ref`` is comparing two real, non-empty, matching lists in the current state ? it is not vacuous `*today*`. The only concern is its robustness against future regression (no independent non-emptiness assertion).

Now let me verify the one final false-confidence angle: the L4a signature guard. Could it pass even if the WINDOW stage `*did*` branch on a profile through some other mechanism (e.g., reading a global/env)? The guard only checks parameter names. The docstring is honest about this (`"structural guard... determinism exercised end-to-end in L4b"`). So L4a alone is weak, but it's explicitly backstopped by L4b. That's an acceptable design, not a flaw ? but worth noting the guard would NOT catch a profile leak via ``os.environ`` or a module global. Given ``trajectory_to_action_windows`` is a pure staticmethod, that risk is low. Let me quickly confirm it reads no globals/env.

37. user (2026-06-21T10:21:00.350Z)

```
335         if fps <= 0:
336             fps = 30.0
337         if frame_width <= 0:
```

```

338         frame_width = 1280.0
339
340         active = TrackNetRunner._active_frames(points, fps, frame_width,
velocity_thresh)
341
342         if not active:
343             return []
344
345         max_gap_frames = int(merge_gap * fps)
346
347         def _finalize(group: List[Tuple[int, float]]) -> Dict[str, Any]:
348             vels = [v for _, v in group]
349             return {
350                 "start": group[0][0] / fps + chunk_start,
351                 "end": group[-1][0] / fps + chunk_start,
352                 "active_frame_count": len(group),
353                 "peak_velocity": max(vels),
354                 "mean_velocity": sum(vels) / len(vels),
355                 "track_id_count": 1, # single shuttle; kept for window-shape parity
356                 "chunk_index": chunk_index,
357             }
358
359         groups: List[List[Tuple[int, float]]] = []
360         group = [active[0]]
361         for af in active[1:]:
362             if af[0] - group[-1][0] <= max_gap_frames:
363                 group.append(af)
364             else:
365                 groups.append(group)
366                 group = [af]
367         groups.append(group)
368
369         if max_window_duration > 0:
370             groups = TrackNetRunner._split_overlong_groups(
371                 groups, int(max_window_duration * fps), int(min_split_gap * fps),
372                 hard_cap=window_hard_cap)
373
374         # R4 ? rally-END inactivity trim (default OFF). Applied AFTER the cap split so
each rally

```

38. assistant (2026-06-21T10:21:04.402Z)

The windowing is purely a function of its arguments (points, fps, thresholds) ? no `os.environ`, no globals. So L4a + L4b together genuinely prove the decoupling. Good.

Let me do a final consolidated full-suite sanity run to confirm nothing else regressed and check the runlog template isn't a test artifact issue, then compile findings.

39. user (2026-06-21T10:21:13.450Z)

```

..... [100%]
59 passed in 6.49s

```

40. assistant (2026-06-21T10:21:23.938Z)

All green. Let me compile my findings. Let me re-read the L4c test once more to nail the exact line for the missing non-emptiness guard.

The findings, grounded:

1. ****L4c (`test\_worker\_intervals\_match\_inprocess`, line 139)**** lacks an independent non-emptiness guard. `assert worker == ref` would pass vacuously if both were `[ ]`. Today both are `[(2.033, 5.967), (9.033, 12.967)]` (I verified), and L4b's `len(local) >= 1` catches a stub regression ? but L4c as a standalone proof relies on L4b co-running. A `process\_video`

early-`return []` (missing API key, bad duration, unresolvable runner ? all real code paths at trajectory_hybrid.py:151/177/187/200) on *both* sides would slip through L4c silently.

2. ****Server `main()` fatal path (main.py:587-591) is untested.**** The symmetric worker path has `test\_worker\_cmd\_infer\_aborts\_rc2\_on\_incoherent\_profile`, but the server's `enforce\_startup\_checks` ? except StartupCheckError ? sys.exit(1) wiring ? the load-bearing "loud error before any job" for the API server ? has no test, and neither does its default-OFF no-op behavior.

These are the two real test-rigor gaps. The `in`-vs-`==` assertions are nits (verified to match singleton sets today). L4a's signature-only guard is acceptable because it's explicitly backstopped by L4b and the function is genuinely pure.

41. user (2026-06-21T10:21:50.447Z)

Structured output provided successfully

42. assistant (2026-06-21T10:22:06.339Z)

I've completed the test-rigor / false-confidence review of the M5 change. Here's the summary of what I verified and found.

****Verified sound (the load-bearing claims hold):****

- The stub trajectory is genuinely identical across all three profile arms ? it's generated purely from `StubConfig` defaults (`num\_rallies=2`), independent of video bytes; `video\_id` only affects the output CSV *filename*, not the rows. I ran it and confirmed every arm yields the same 2 non-empty segments `[(2.033, 5.967), (9.033, 12.967)]`.
- L4b's `len(local) >= 1` is a real, non-vacuous guard ? I confirmed it fires on the all-empty case, so `local == mock == none` is comparing two specific 2-tuple lists, not `[] == []`.
- L4c's worker run produces the identical 2 ranges (I ran `cmd\_infer` directly), so `worker == ref` is comparing real matching data today, not two empties.
- L4a's signature guard correctly excludes the real params and the windowing function is genuinely pure (no env/globals).
- DEFAULT-OFF: `AppConfig` default `profile=''` ? `run\_startup\_checks` returns `[]` immediately; all 25 new tests + 59 with deployment config pass.

****Real gaps found (both minor):****

1. ****L4c lacks an independent non-emptiness guard**** (test_profile_parity.py:139). `assert worker == ref` would pass vacuously if a stub/windowing regression zeroed both sides; `process\_video` has multiple real `return []` paths. It relies on L4b co-running to catch emptiness. One-line fix: add `assert ref`.
2. ****The server `main()` fatal-abort path is untested**** (backend/main.py:587-591). The worker's symmetric rc-2 abort is covered, but the API server's `enforce\_startup\_checks` ? sys.exit(1) ? the other half of the "loud error before any job" contract ? and its default-OFF no-op have zero coverage.

Plus two nits: four assertions use `in` instead of `==` (tolerating spurious extra checks, though they match singletons today), and the L4a name-only guard is acceptable-as-backstopped.

Relevant files: `C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test\_profile\_parity.py`, `C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test\_startup\_checks.py`, `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py`.